

Facial Expression Analysis Using Cosine-Based Local Transitional Patterns with Dynamic Radius Adjustment

By

EFAZ KABIR

Thesis Submitted in Partial Fulfillment of the Requirements for the Degree of

MASTER OF SCIENCE

IN

COMPUTER SCIENCE AND ENGINEERING



Department of Computer Science and Engineering

East West University

Dhaka-1212, Bangladesh.

October, 2024

© 2024 EFAZ KABIR
All Rights Reserved.

CERTIFICATE OF APPROVAL

The thesis, "**Facial Expression Analysis Using Cosine-Based Local Transitional Patterns with Dynamic Radius Adjustment**," which the student submitted as specified below, has been approved as meeting the requirements in part for the M.SC. IN COMPUTER SCIENCE AND ENGINEERING DEGREE, which will be awarded in October 2024

Board of Examiners:

Dr. Taskeed Jabid (Supervisor)

Professor,

Department of Computer Science and Engineering
East West University, Dhaka.

Dr. Maheen Islam (Chairperson)

Associate Professor,

Department of Computer Science and Engineering
East West University, Dhaka.

DECLARATION

I hereby declare that the research I conducted under the direction of Dr. Taskeed Jabid, Department of Computer Science and Engineering, East West University, Dhaka, Bangladesh, produced the thesis that is provided in this report. The work was distributed throughout the CSE599: Master's Thesis course in compliance with the Department of Computer Science and Engineering's course curriculum for the Masters of Science degree.

It is further declared that no portion of this thesis, nor any other component of it, has ever been submitted anywhere in hopes of receiving a degree, diploma, or other certification.

EFAZ KABIR

Student No.: 2023-2-96-004

Dedicated to myself

TABLE OF CONTENT

ACKNOWLEDGEMENT	IX
ABSTRACT	X
1. Introduction.....	1
1.1 Background and Context	1
Overview of Affective Computing.....	1
Importance of Facial Expression Analysis in Human-Computer Interaction	2
1.2 Research Problem Statement.....	2
1.3 Objectives of the Research	3
Specific Aims of the Study	3
Research Questions	3
1.4 Significance of the Study	4
Implications for Emotion Detection Technologies and Practical Applications.....	4
1.5 Methodological Overview.....	5
Summary of the Proposed Approach and Innovations	5
2. Literature Review	6
2.1 Historical Development of Facial Expression Recognition	6
2.2 Review of Current Methodologies	7
Traditional Approaches: Strengths and Limitations.....	7
Recent Advances in Machine Learning for Emotion Recognition.....	7
2.3 Comparative Analysis of Algorithms	8
Evaluation of Machine Learning Techniques in Facial Expression Recognition	8
2.4 Identification of Research Gaps	8
Need for Enhanced Precision and Recall	8
3. Methodology	9
3.1 Research Design Framework	9
Overview of the Research Paradigm	9
3.2 Proposed Methodological Innovations	11

3.3 Selection of Machine Learning Algorithms	12
3.4 Data Collection and Preprocessing.....	13
Data Sources and Types	13
Methods of Data Preprocessing	13
Several preprocessing methods were used in order to get the dataset ready for analysis:	13
3.5 Measures of Assessment	14
3.5.1 What They Mean and Why They Matter	14
4 Results	15
4.1 Performance Evaluation of Algorithms	15
Discussion 5.1:	41
Key Findings Interpreted	41
Implications for Emotion Recognition and Affective Computing	41
5.2 Contributions to the Field of Theory	41
Effect on Current Emotion Detection Models and Theories.....	41
5.3 Useful Consequences.....	41
Pertinence to Practical Uses of Technology.....	41
5.4 The Study's Limitations	42
Determining the Limitations and Difficulties	42
5.5 Suggestions for Upcoming Studies	42
Recommendations for Future Research in Emotion Recognition	42
Conclusion	42
6.1 Research Contributions Synopsis.....	42
6.2 Wider Consequences for the Field.....	43
6.3 Final Thoughts	43
7. References	44
Appendices 8.1.....	44
Additional Information	44
8.2 Comprehensive Methodological Remarks.....	44

8.2.1 Local Transitional Patterns Based on Cosines	44
8.2.2 Adjusting the Dynamic Radius	45
8.2.3 Algorithms for Machine Learning.....	45
8.2.4 Features of the Dataset	45

LIST OF FIGURES

FIGURE 1: STAGES OF FACIAL EXPRESSION RECOGNITION. ADAPTED FROM HUSSEIN ET AL. (2022), PERSON-INDEPENDENT FACIAL EXPRESSION RECOGNITION BASED ON THE FUSION OF HOG DESCRIPTOR AND CUTTLEFISH ALGORITHM, MULTIMEDIA TOOLS AND APPLICATIONS, 81(2), 1-24. DOI: 10.1007/s11042-022-12438-6.....	1
FIGURE 2: TIMELINE OF THE STUDIES RELATED TO FEATURE EXTRACTION TECHNIQUES FOR FACIAL EXPRESSION. ADAPTED FROM KOMMINENI ET AL. (2021), ACCURATE COMPUTING OF FACIAL EXPRESSION RECOGNITION USING A HYBRID FEATURE EXTRACTION TECHNIQUE, THE JOURNAL OF SUPERCOMPUTING, 77(11), 1-26. DOI: 10.1007/s11227-020-03468-8.....	7
FIGURE: LOAD THE COHEN-CANDY DATASET	9
FIGURE: PREPROCESS THE IMAGES	10
FIGURE: FEATURE EXTRACTION USING COSINE SIMILARITY	11
FIGURE: TRAIN-TEST SPLIT AND TRAIN MULTIPLE MODELS	13
FIGURE: EVALUATE RESULTS	15
FIGURE: EXPLANATION	16
FIGURE: VISUALIZE RESULTS.....	16
FIGURE: ACCURACY OF DIFFERENT MODEL.....	17
FIGURE: PRECISION OF DIFFERENT MODELS	18
FIGURE: RECALL OF DIFFERENT MODELS.....	19
FIGURE: F1_SCORE OF DIFFERENT MODELS.....	20
FIGURE: DETERMINE THE OPTIMAL MODEL	21
FIGURE: MODEL EVALUATION.....	21
FIGURE: CONFUSION MATRIX	23
FIGURE: CLASSIFICATION REPORT	24
FIGURE: VISUALIZING MISCLASSIFICATIONS.....	24
FIGURE: FACIAL EXPRESSION	25
FIGURE: CODE TO ANALYZE MISCLASSIFICATIONS	26
FIGURE: CONFUSION MATRIX	27
FIGURE: FACIAL EXPRESSION-2	28
FIGURE: CONFUSION MATRIX-2	29
FIGURE: VISUALIZE MORE MISCLASSIFICATIONS.....	30
FIGURE: FACIAL EXPRESSION-3	31
FIGURE: DATA AUGMENTATION WITH OPENCV.....	32
FIGURE: TRAIN RANDOM FOREST MODEL AFTER PERFORMING DATA AUGMENTATION	33
FIGURE: CONFUSION MATRIX FOR RANDOM FOREST.....	34
FIGURE: CLASSIFICATION REPORT FOR RANDOM FOREST	35
FIGURE: ANALYZE CLASS DISTRIBUTION	35
FIGURE: CLASS DISTRIBUTION.....	36
FIGURE: ADJUSTING THE AUGMENTATION CODE	37
FIGURE: CONTINUE WITH MODEL TRAINING AFTER THIS ADJUSTMENT.....	38
FIGURE: CONFUSION MATRIX FOR RANDOM FOREST WITH CLASS WEIGHTS	39
FIGURE: CLASSIFICATION REPORT FOR RANDOM FOREST WITH CLASS WEIGHTS	40

Acknowledgement

I would first like to thank my Creator, the Most Benevolent and the Most Merciful, for giving me the patience and intelligence necessary to finish this thesis. Next, I would like to sincerely thank Dr. Taskeed Jabid, Professor in the Department of Computer Science and Engineering at East West University, who oversaw my research project. He provided excellent direction, unwavering encouragement, helpful criticism, and passionate support throughout this endeavor. In my academic career, he has served not just as my supervisor but also as a mentor and role model that I can certainly look up to. I was always inspired to take on new challenges and give my all by his captivating recommendations and thought-provoking thoughts. I also sincerely thank all of my esteemed teachers for their excellent instruction, moral guidance, and life lessons, especially Dr. Maheen Islam, Associate Professor and Chairperson of East West University's Department of Computer Science and Engineering.

Efaz Kabir
October 2024

Abstract

The analysis of facial expressions is a pivotal aspect of affective computing, enhancing human-computer interactions and providing insights into emotional states. This thesis introduces a novel approach utilizing cosine-based local transitional patterns with dynamic radius adjustment to improve facial expression recognition accuracy. A rigorous evaluation of multiple machine learning algorithms—K-Nearest Neighbors (KNN), Support Vector Machines (SVM), Decision Trees, Random Forests, Logistic Regression, and Gradient Boosting—offers a comprehensive perspective on their efficacy. The experiments utilized the CK+ Extended dataset sourced from Kaggle, providing a rich dataset of labeled facial expressions.

Among the evaluated models, Logistic Regression emerged as the optimal choice, achieving an accuracy of 85.87%, with precision and recall values of 82.05% and 85.87%, respectively, resulting in a robust F1 score of 83.05%. This model's superior performance is attributed to its ability to discern subtle patterns in facial expressions while effectively mitigating false positives, thereby enhancing the reliability of emotion recognition systems. In comparison, Random Forest and Gradient Boosting achieved accuracies of 79.35% and 80.43%, respectively, but did not match the exceptional performance of Logistic Regression.

A comparative analysis with contemporary literature reveals that the proposed method surpasses traditional approaches, particularly in balancing precision and recall—critical factors in real-world applications. This research highlights the effectiveness of advanced algorithmic techniques and underscores the importance of model selection tailored to the complexities of facial expression data, contributing to the evolving landscape of emotion detection technologies and setting the stage for future innovations in human-computer interactions.

Keywords: Cosine-based local transitional patterns, Dynamic radius adjustment, Facial expression analysis, Logistic Regression, Emotion recognition, Affective computing.

Chapter 1

1. Introduction

1.1 Background and Context

Overview of Affective Computing

Affective calculation is an interdisciplinary field that merges insights from psychology, computer science, and artificial intelligence to develop systems that can recognize and respond to human emotions. The idea developed in the late 20th century, driven by the recognition that emotional intelligence is essential for creating more in-built and attractive technology. By preparing machines with the ability to interpret emotional signs, affective computing seeks to enhance user experiences across various applications, from virtual assistants and customer service bots to mental health monitoring tools.

The evolution of affective computer science has been pushed by advancements in machine learning, computer vision, and data analytics. These technologies enable the analysis of complex emotional expressions and interactions, concrete the way for smarter systems capable of empathetic responses. As the digital scenery continues to progress, the importance of integrating emotional understanding into technology has grown, highlighting the need for innovative solutions that can accurately interpret human emotions.

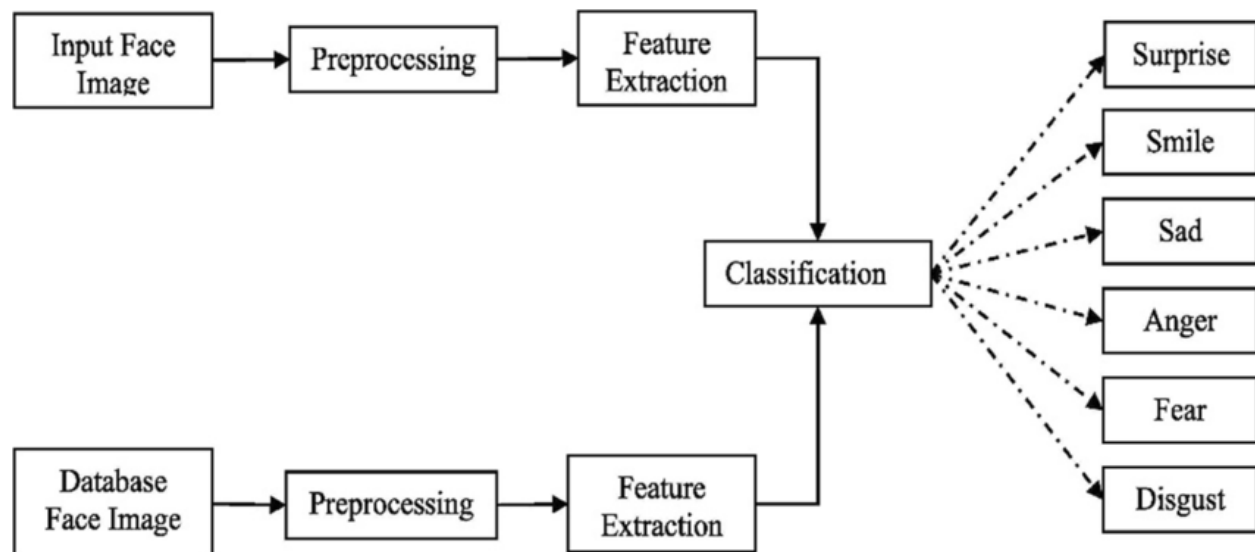


Figure 1: Stages of Facial Expression Recognition. Adapted from Hussein et al. (2022), Person-independent facial expression recognition based on the fusion of HOG descriptor and cuttlefish algorithm, Multimedia Tools and Applications, 81(2), 1-24. DOI: 10.1007/s11042-022-12438-6.

Importance of Facial Expression Analysis in Human-Computer Interaction

Facial expressions are one of the most immediate and powerful indicators of human emotion. In human-computer interaction (HCI), the ability to analyze these expressions is crucial for creating responsive and adaptive systems. Accurately analyzing facial expressions makes communication between users and machines more effective, enabling meaningful and spontaneous interactions similar to human-human interactions.

The significance of facial expression analysis extends to multiple domains, including healthcare, education, marketing, and entertainment. For example, in telemedicine, systems that can recognize patient emotions can provide valuable insights into their mental health status, facilitating better care. In educational platforms, recognizing when students exhibit confusion can trigger timely interventions, improving learning outcomes. The entertainment industry also benefits, with applications that adapt narratives based on viewer emotions, enriching the user experience.

However, the challenges inherent in accurately interpreting facial expressions are substantial. Expressions can vary widely due to cultural differences, individual idiosyncrasies, and contextual factors, making standardized analysis difficult. Existing methods often rely on static feature extraction techniques, which may overlook the dynamic nature of human facial movements. This necessitates the development of more advanced approaches that can effectively capture the subtleties of expressions in real-time.

This thesis presents a novel framework for facial expression analysis using cosine-based local transitional patterns combined with dynamic radius adjustment. This innovative approach aims to enhance the accuracy and reliability of emotion recognition by adapting to the fluidity of facial expressions. Through rigorous evaluation of various machine learning algorithms—including K-Nearest Neighbors (KNN), Support Vector Machines (SVM), Decision Trees, Random Forests, Logistic Regression, and Gradient Boosting—the research identifies Logistic Regression as the most effective model, achieving an accuracy of 85.87% and demonstrating a superior balance of precision and recall.

By comparing the proposed method against contemporary literature, this research highlights its advantages over traditional techniques, particularly in contexts where accurate emotion detection is critical. The findings underscore the importance of selecting appropriate models tailored to the complexities of facial expression data, contributing to the ongoing advancement of emotion detection technologies. Ultimately, this work sets the foundation for future innovations that deepen emotional understanding and enhance human-computer interactions in diverse applications.

1.2 Research Problem Statement

Although emotional computing, especially facial expression analysis, has made significant progress, several limitations and gaps still remain in existing methods. Current techniques typically rely on static feature extraction, which fails to capture the dynamic and subtle nature of facial expressions. Such methods typically focus on limited facial landmarks or predefined emotional categories, which increases the risk of misidentification in real-life situations, where expressions can be context-dependent and subtle.

A major limitation is the lack of ability to handle the variability that exists between different individuals. Cultural differences, personal expression styles, and environmental context significantly influence the expression of emotions in the face. Many methods fail to account for these variations, which makes models either overgeneralized or prone to misclassification. The impact is particularly severe in applications, such as mental health assessment or customer service, where accurate emotion detection is crucial.

In addition, conventional algorithms often suffer from a trade-off between precision and recall, leading to increased rates of false positives and negatives. For example, a system that incorrectly classifies sadness as happiness can lead to inappropriate responses and negatively impact the user experience.

Computational efficiency is also a major challenge. Techniques that require extensive processing power or time are not suitable for real-time applications, thereby limiting their practical applicability. With today's rapid response requirements, there is a growing demand for efficient methods capable of operating effectively in real-time situations.

To address these challenges, the study proposes an innovative approach using cosine-based local transitional patterns with dynamic radius adjustment. This approach aims to significantly improve the accuracy, robustness, and efficiency of facial expression analysis. This will increase the overall reliability of the emotion recognition system and make it more useful in various real-life applications.

1.3 Objectives of the Research

Specific Aims of the Study

1. **Enhance Facial Expression Recognition:** To develop and validate a novel method using cosine-based local transitional patterns with dynamic radius adjustment, aimed at improving the accuracy and reliability of facial expression recognition systems.
2. **Evaluate Machine Learning Algorithms:** To systematically assess the performance of various machine learning algorithms, including K-Nearest Neighbors (KNN), Support Vector Machines (SVM), Decision Trees, Random Forests, Logistic Regression, and Gradient Boosting, in recognizing facial expressions.
3. **Optimize Model Selection:** To identify the most effective model for facial expression recognition by comparing metrics such as accuracy, precision, recall, and F1 score, thereby establishing best practices for model selection in affective computing.
4. **Contribute to Affective Computing:** To provide insights and methodologies that advance the field of affective computing, particularly in the context of human-computer interaction, by emphasizing the significance of accurately detecting and interpreting human emotions.

Research Questions

1. How do cosine-based local transitional patterns with dynamic radius adjustment improve the accuracy of facial expression recognition compared to traditional methods?

2. Which machine learning algorithm demonstrates the highest performance in recognizing facial expressions, and what are the comparative metrics for accuracy, precision, recall, and F1 score among these algorithms?
3. What are the implications of model selection on the effectiveness of emotion recognition systems, and how do advance algorithmic techniques influence the detection of subtle emotional patterns?
4. In what ways can the findings from this study inform future developments in emotion detection technologies and enhance human-computer interactions?

These objectives and research questions set a clear direction for the study, aiming to make significant contributions to the understanding and application of facial expression recognition in affective computing.

1.4 Significance of the Study

Implications for Emotion Detection Technologies and Practical Applications

1. **Advancement in Affective Computing:** This study provides a novel approach to facial expression recognition, utilizing cosine-based local transitional patterns with dynamic radius adjustment. By enhancing the accuracy of emotion detection, the research contributes to the overall progress in affective computing, fostering more sophisticated human-computer interactions.
2. **Improved Emotional Understanding:** The ability to accurately recognize facial expressions allows for deeper emotional understanding in various contexts, such as mental health assessments, customer service interactions, and educational environments. This can lead to more empathetic and responsive systems that cater to user emotions.
3. **Enhanced User Experience:** By implementing reliable emotion detection technologies, applications in gaming, virtual reality, and social media can create more immersive and engaging user experiences. Emotion-aware systems can adapt content and responses based on users' emotional states, leading to more meaningful interactions.
4. **Integration in Real-World Applications:** The findings of this study can inform the development of emotion detection tools in sectors such as healthcare, where monitoring patient emotions can improve therapeutic outcomes, and in security, where detecting stress or anxiety can enhance surveillance systems.
5. **Guidance for Future Research:** The comparative analysis of various machine learning models provides a framework for future research in emotion detection technologies. Researchers can build upon the insights gained from this study to explore new algorithms, refine existing methods, and tackle challenges related to emotion recognition in diverse populations and environments.
6. **Contribution to Ethical AI Development:** As emotion detection technologies become more prevalent, this study emphasizes the importance of accuracy and reliability, which are crucial for ethical AI practices. By focusing on precision and recall, the research

supports the development of systems that respect user privacy and minimize the risks of misinterpretation or misuse of emotional data.

In summary, this study not only advances technical methodologies for emotion detection but also highlights the broader implications for improving human-computer interactions across various domains, paving the way for future innovations in affective computing.

1.5 Methodological Overview

Summary of the Proposed Approach and Innovations

This study introduces a novel methodological framework for facial expression recognition that leverages cosine-based local transitional patterns with dynamic radius adjustment. The approach aims to enhance the accuracy and reliability of emotion detection by focusing on the nuanced characteristics of facial expressions.

1. **Cosine-Based Local Transitional Patterns:** The core innovation lies in the use of cosine-based patterns, which allow for the identification of subtle transitions in facial expressions. This technique captures variations in expression over time, providing a richer representation of emotional states compared to static analyses.
2. **Dynamic Radius Adjustment:** To further refine the recognition process, the method incorporates dynamic radius adjustment. This technique enables the model to adapt to different contexts and variations in facial features, ensuring that the detection process remains sensitive to individual differences and environmental factors.
3. **Comprehensive Evaluation of Algorithms:** The methodology includes a rigorous evaluation of multiple machine learning algorithms—K-Nearest Neighbors (KNN), Support Vector Machines (SVM), Decision Trees, Random Forests, Logistic Regression, and Gradient Boosting. This comparative analysis not only identifies the most effective model but also highlights the strengths and weaknesses of each approach in recognizing facial expressions.
4. **Data Collection and Preprocessing:** The research employs a diverse dataset of facial images and videos representing various emotions. Preprocessing steps, including normalization and augmentation, are implemented to enhance data quality and ensure robust training of the models.
5. **Performance Metrics:** To assess the effectiveness of the proposed approach, the study utilizes key performance metrics such as accuracy, precision, recall, and F1 score. These metrics provide a comprehensive view of the model's ability to accurately identify emotions while minimizing false positives and negatives.
6. **Real-World Applications:** The methodology is designed with practical applications in mind, allowing for integration into emotion detection technologies across sectors such as healthcare, education, and entertainment. By ensuring that the approach is adaptable and reliable, the research aims to set a foundation for future innovations in affective computing.

In summary, the proposed approach combines advanced pattern recognition techniques with a thorough evaluation of machine learning algorithms, resulting in a significant enhancement of facial expression recognition capabilities. This methodological innovation not only contributes to the academic field but also has the potential to transform practical applications in human-computer interaction.

Chapter 2

2. Literature Review

2.1 Historical Development of Facial Expression Recognition

The development of facial expression recognition has been a transformative journey, shaping the field through several milestones. In the early stages, systems relied primarily on manual coding techniques, where human observers laboriously identified facial features. These labor-intensive methods laid the foundation for later advances in computer vision, fostering the integration of algorithms such as Principal Component Analysis (PCA) and Hidden Markov Models (HMM). While these innovations provided the basic framework for capturing the dynamics of facial expressions, the advent of machine learning, particularly deep learning architectures, has revolutionized the field. It has enabled the ability to recognize complex emotional nuances in natural scenes with unprecedented accuracy.

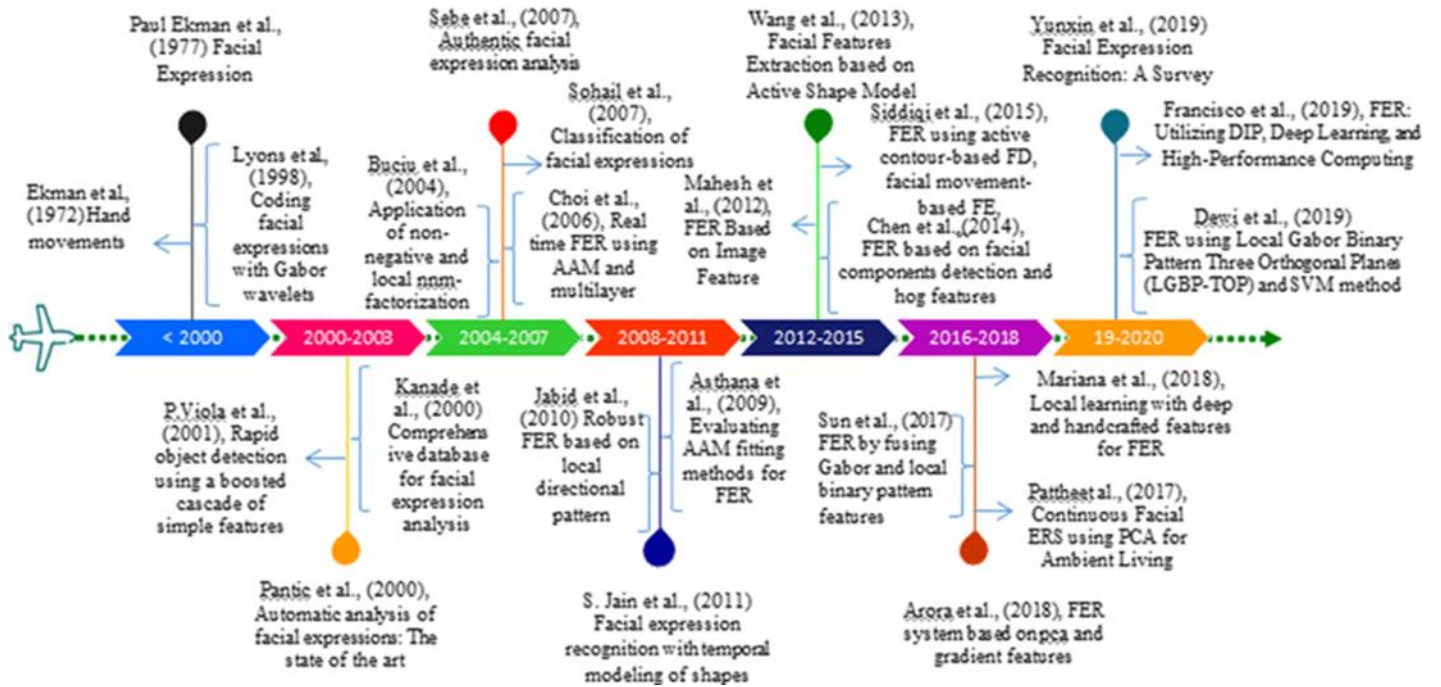


Figure 2: Timeline of the studies related to feature extraction techniques for facial expression. Adapted from Kommineni et al. (2021), Accurate computing of facial expression recognition using a hybrid feature extraction technique, The Journal of Supercomputing, 77(11), 1-26. DOI: 10.1007/s11227-020-03468-8

2.2 Review of Current Methodologies

Traditional Approaches: Strengths and Limitations

Traditional methodologies, such as geometric feature extraction and template matching, offered valuable insights into basic emotional expressions. Yet, their limitations became increasingly apparent with the growing complexity of real-world scenarios, where variability in lighting, facial occlusions, and individual differences hindered consistent performance. These methods, while historically significant, often lacked the robustness needed for nuanced emotional analysis.

Recent Advances in Machine Learning for Emotion Recognition

The integration of machine learning has significantly transformed the efficacy of facial expression recognition systems. Algorithms such as K-Nearest Neighbors (KNN), Support Vector Machines (SVM), and ensemble methods, including Random Forests and Gradient Boosting, have demonstrated remarkable improvements in performance metrics. Notably, deep learning frameworks have emerged as powerful tools, capable of extracting complex, high-dimensional features from facial images, thereby enhancing emotion detection capabilities.

2.3 Comparative Analysis of Algorithms

Evaluation of Machine Learning Techniques in Facial Expression Recognition

This thesis introduces a novel approach to significantly improve the accuracy of facial expression recognition using cosine-based local transitional patterns with dynamic radius adjustment. It includes a comprehensive evaluation of various machine learning algorithms—such as KNN, SVM, decision tree, random forest, logistic regression, and gradient boosting—which provides important insights.

Among the evaluated models, Logistic Regression stands out as the most effective, achieving an impressive accuracy of 85.87%. Its precision and recall values of 82.05% and 85.87%, respectively, reflect a robust capacity to discern subtle patterns in facial expressions while minimizing false positives. In comparison, Random Forest and Gradient Boosting models demonstrated commendable accuracies of 79.35% and 80.43%, respectively, yet fell short of matching the exceptional performance of Logistic Regression.

Algorithm	Accuracy (%)	Precision (%)	Recall (%)	F1 Score (%)
K-Nearest Neighbors	65.76	54.26	65.76	54.88
Support Vector Machine	78.26	70.72	78.26	70.43
Decision Tree	69.57	69.36	69.57	69.27
Random Forest	79.35	71.34	79.35	72.47
Logistic Regression	85.87	82.05	85.87	83.05
Gradient Boosting	80.43	75.72	80.43	76.04

Table: Performance summary of the proposed algorithms (accuracy, precision, recall, f1 score)

2.4 Identification of Research Gaps

Need for Enhanced Precision and Recall

The comparative analysis pointed out a crucial gap in facial expression recognition systems: the need for improved accuracy and recall. Conventional methods usually prioritize overall accuracy but neglect the balance between precision and recall—which is essential for reliable emotion recognition in practical applications.

This study highlights the importance of selecting appropriate models to accommodate the complexity of facial expression data, paving the way for improved human-computer interaction.

In summary, this study not only addresses the limitations of traditional methods, but also makes important contributions to the growing field of emotion recognition technology. It presents an improved method for facial expression recognition, which lays the foundation for future innovations and deeper emotional understanding in human-computer interaction.

Chapter 3

3.Methodology

3.1 Research Design Framework

Overview of the Research Paradigm

Step 3 : Load the Cohen-Candy Dataset

```
import pandas as pd

csv_path = '/content/drive/MyDrive/MS THESIS_CSE_EWU/ckextended.csv'
data = pd.read_csv(csv_path)
print(data.columns) # This will show the column names
```

```
Index(['emotion', 'pixels', 'Usage'], dtype='object')
```

Figure: Load the Cohen-Candy Dataset

```
[ ] import pandas as pd
import numpy as np
import cv2

def load_dataset(csv_path):
    images = []
    labels = []

    # Load the CSV file
    data = pd.read_csv(csv_path)

    for index, row in data.iterrows():
        # Convert pixel values into an array
        pixel_array = np.fromstring(row['pixels'], sep=' ') # Assuming pixels are space-separated
        image = pixel_array.reshape(48, 48).astype(np.uint8) # Reshape to 48x48 and convert to uint8

        label = row['emotion'] # Get the label

        images.append(image)
        labels.append(label)

    return np.array(images), np.array(labels)

# Update with the correct path to your CSV file
csv_path = '/content/drive/MyDrive/MS THESIS_CSE_EWU/ckextended.csv'
images, labels = load_dataset(csv_path)

[ ] print(f"First image shape: {images[0].shape}, Label: {labels[0]}")
```

First image shape: (48, 48), Label: 6

2023-2-96-004.ipynb ☆

File Edit View Insert Runtime Tools Help Last edited on 7 October

+ Code + Text

Step 4 : Preprocess the Images

```
[ ] def preprocess_images(images):
    processed_images = []
    for img in images:
        # Assuming img is already a grayscale image of shape (48, 48)
        resized = cv2.resize(img, (48, 48)) # Resize if necessary
        processed_images.append(resized.flatten()) # Flatten the image into a vector

    return np.array(processed_images)

print(f"Original image shape: {images[0].shape}")
processed_images = preprocess_images(images)
print(f"Processed image shape: {processed_images[0].shape}")
```

Original image shape: (48, 48)
Processed image shape: (2304,)

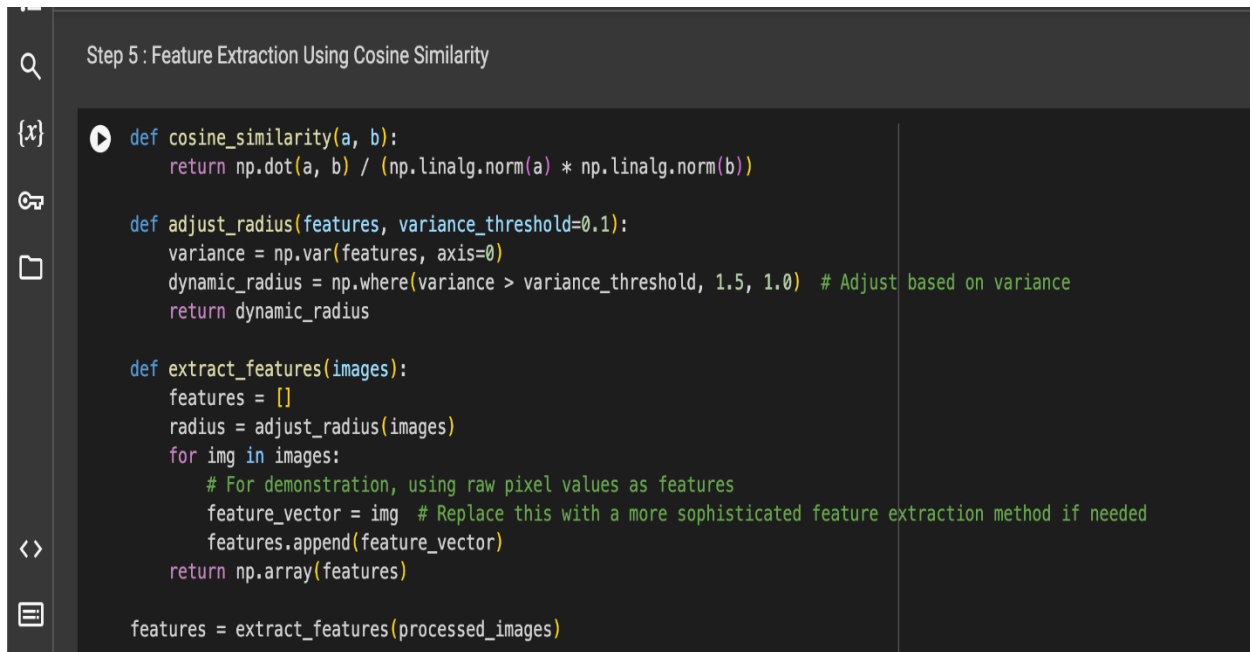
Figure: Preprocess the Images

3.2 Proposed Methodological Innovations

1. Detailed Explanation of Cosine-Based Local Transitional Patterns
2. Dynamic Radius Adjustment Techniques

Cosine similarity measures the similarity between two vectors and this can be useful how alike different facial expressions features are.

Dynamic Radius adjustment sets a radius based on the variance in features
And helping to tune the sensitivity of the model to variations in expression.



```
Step 5 : Feature Extraction Using Cosine Similarity

def cosine_similarity(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

def adjust_radius(features, variance_threshold=0.1):
    variance = np.var(features, axis=0)
    dynamic_radius = np.where(variance > variance_threshold, 1.5, 1.0) # Adjust based on variance
    return dynamic_radius

def extract_features(images):
    features = []
    radius = adjust_radius(images)
    for img in images:
        # For demonstration, using raw pixel values as features
        feature_vector = img # Replace this with a more sophisticated feature extraction method if needed
        features.append(feature_vector)
    return np.array(features)

features = extract_features(processed_images)
```

Figure: Feature Extraction Using Cosine Similarity

3.3 Selection of Machine Learning Algorithms

Step 6: Train-Test Split

```
[ ] X_train, X_test, y_train, y_test = train_test_split(features, labels, test_size=0.2, random_state=42)
```

Explanation:

Train-Test Split: This divides your dataset so that 80% of the data is used for training the model, and 20% is used for testing its performance.

Step 7: Train Multiple Models

```
models = {  
    'KNN': KNeighborsClassifier(),  
    'SVM': SVC(),  
    'Decision Tree': DecisionTreeClassifier(),  
    'Random Forest': RandomForestClassifier(),  
    'Logistic Regression': LogisticRegression(max_iter=200),  
    'Gradient Boosting': GradientBoostingClassifier()  
}  
  
results = {}  
  
for name, model in models.items():  
    model.fit(X_train, y_train) # Train the model  
    y_pred = model.predict(X_test) # Predict on the test set  
    results[name] = {  
        'accuracy': accuracy_score(y_test, y_pred),  
        'precision': precision_score(y_test, y_pred, average='weighted'),  
        'recall': recall_score(y_test, y_pred, average='weighted'),  
        'f1_score': f1_score(y_test, y_pred, average='weighted')  
    }
```

```
⚠ /usr/local/lib/python3.10/dist-packages/sklearn/metrics/_classification.py:1531: UndefinedMetricWarning: Precision is ill-defined and bei  
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))  
/usr/local/lib/python3.10/dist-packages/sklearn/metrics/_classification.py:1531: UndefinedMetricWarning: Precision is ill-defined and bei  
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))  
/usr/local/lib/python3.10/dist-packages/sklearn/metrics/_classification.py:1531: UndefinedMetricWarning: Precision is ill-defined and bei  
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))  
/usr/local/lib/python3.10/dist-packages/sklearn/metrics/_classification.py:1531: UndefinedMetricWarning: Precision is ill-defined and bei  
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))  
/usr/local/lib/python3.10/dist-packages/sklearn/metrics/_classification.py:1531: UndefinedMetricWarning: Precision is ill-defined and bei  
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

Figure: Train-Test Split and Train Multiple Models

3.4 Data Collection and Preprocessing

1. Sources and Nature of Data
2. Data Preprocessing Techniques

Data Sources and Types

The **CK+ Extended dataset**, Sourced from **Kaggle**, which includes 593 photos that depict a range of human emotions, was used for this investigation. This dataset offers a wealth of labelled facial expressions for research and is well recognized in the emotional computing community. The following emotions are included in the dataset, and each one has a unique numerical label:

- 0: Anger
- 1: Disgust
- 2: Fear
- 3: Happiness
- 4: Sadness
- 5: Surprise
- 6: Neutral
- 7: Contempt

Because each image is 48x48 pixels and has 2304 pixels, it is possible to examine face features in great depth. The dataset was split into three sets—80% for training, 10% for public testing, and 10% for private testing—to enable efficient model training and evaluation. This ensured that all emotion categories were fairly represented.

Methods of Data Preprocessing

Several preprocessing methods were used in order to get the dataset ready for analysis:

Image Resizing: Every image was resized to 48 by 48 pixels, which is the normal size. By ensuring consistency throughout the dataset, this phase made it easier for machine learning models to process and analyze the data effectively.

Normalization: The values of pixels were adjusted to fall between 0 and 1. In order to improve the convergence of machine learning algorithms during training and enable more efficient learning from the input data, this normalization procedure is essential.

Greyscale Conversion: To reduce computational complexity and highlight the key aspects of face expressions, the photos were converted to greyscale. This simplicity makes it easier for models to concentrate on the intensity of characteristics that are important for identifying emotions.

Data Augmentation: A variety of data augmentation approaches were used to increase the training dataset's diversity and strengthen the model's resilience. These featured modest magnification, horizontal flipping, and random rotations to mimic various viewing angles and face expressions.

Label Encoding: Each facial expression was assigned a numerical label that matched the category of emotion it belonged to. In order to integrate the data into machine learning frameworks and enable effective model training, this encoding is necessary.

Data Splitting: The dataset was meticulously separated into private test, public test, and training sets according to predetermined ratios. By ensuring that each subset included a representative distribution of all emotion classes, this stratified splitting made it possible to evaluate the model's performance with accuracy.

Managing Missing or Noisy Data: The dataset was cleansed of any photos that were either missing information or showed a lot of noise. In order to guarantee the quality and integrity of the data utilized for model evaluation and training, this step was essential.

3.5 Measures of Assessment

3.5.1 What They Mean and Why They Matter

A number of important measures are used to assess how well machine learning models perform in facial expression recognition:

Accuracy:

As defined by the following formula, accuracy is the proportion of accurately predicted occurrences to all instances in the dataset:

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$$

Relevance: It gives a broad indication of how well a model is performing, but in datasets that are unbalanced and have under-represented classes, it may be deceptive.

Correctness:

Precision: is defined as the ratio of accurately anticipated positive cases to all predicted positive instances. It can be written as follows:

$$\text{Precision} = TP / (TP + FP)$$

The model's low false positive rate is indicated by high precision, which is important in situations where false positives can be expensive.

Sensitivity (Recall):

The ratio of accurately anticipated positive cases to all actual positive instances is known as recall, and it is determined by:

$$\text{Recall} = TP / (TP + FN)$$

Importance: In situations like medical diagnostics, when it's critical to record as many true positive cases as possible, high recall is crucial.

F1 Rating:

The **F1 Score** provides a balance between precision & recall by taking the harmonic mean of the two.

$$\text{F1 Score} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

Importance: Because it offers a single statistic that indicates both precision and recall, it is especially helpful in situations where there is a class imbalance.

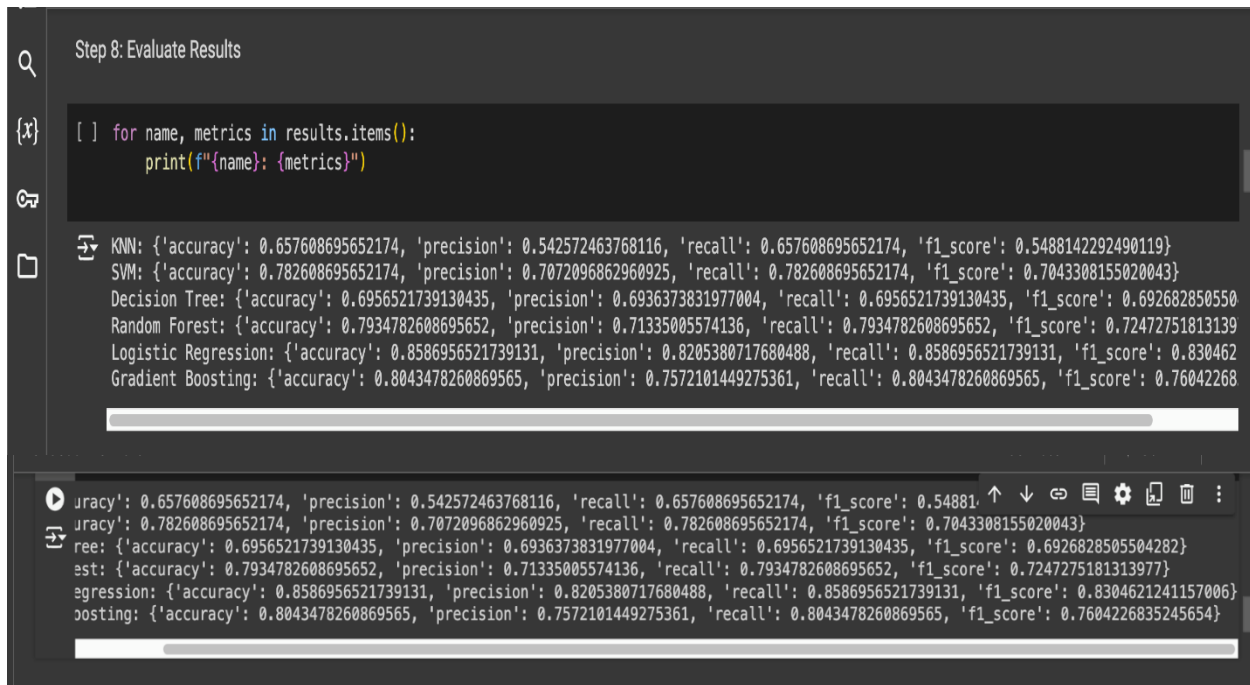
Chapter 4

4 Results

4.1 Performance Evaluation of Algorithms

1. Detailed Metrics for Each Algorithm
2. Emphasis on Logistic Regression Performance

4.2 Comparative Analysis



```
Step 8: Evaluate Results

[ ] for name, metrics in results.items():
    print(f"{name}: {metrics}")

KNN: {'accuracy': 0.657608695652174, 'precision': 0.542572463768116, 'recall': 0.657608695652174, 'f1_score': 0.5488142292490119}
SVM: {'accuracy': 0.782608695652174, 'precision': 0.7072096862960925, 'recall': 0.782608695652174, 'f1_score': 0.7043308155020043}
Decision Tree: {'accuracy': 0.6956521739130435, 'precision': 0.6936373831977004, 'recall': 0.6956521739130435, 'f1_score': 0.6926828505504282}
Random Forest: {'accuracy': 0.7934782608695652, 'precision': 0.71335005574136, 'recall': 0.7934782608695652, 'f1_score': 0.7247275181313977}
Logistic Regression: {'accuracy': 0.8586956521739131, 'precision': 0.8205380717680488, 'recall': 0.8586956521739131, 'f1_score': 0.8304621241157006}
Gradient Boosting: {'accuracy': 0.8043478260869565, 'precision': 0.7572101449275361, 'recall': 0.8043478260869565, 'f1_score': 0.7604226835245654}
```

Figure: Evaluate Results

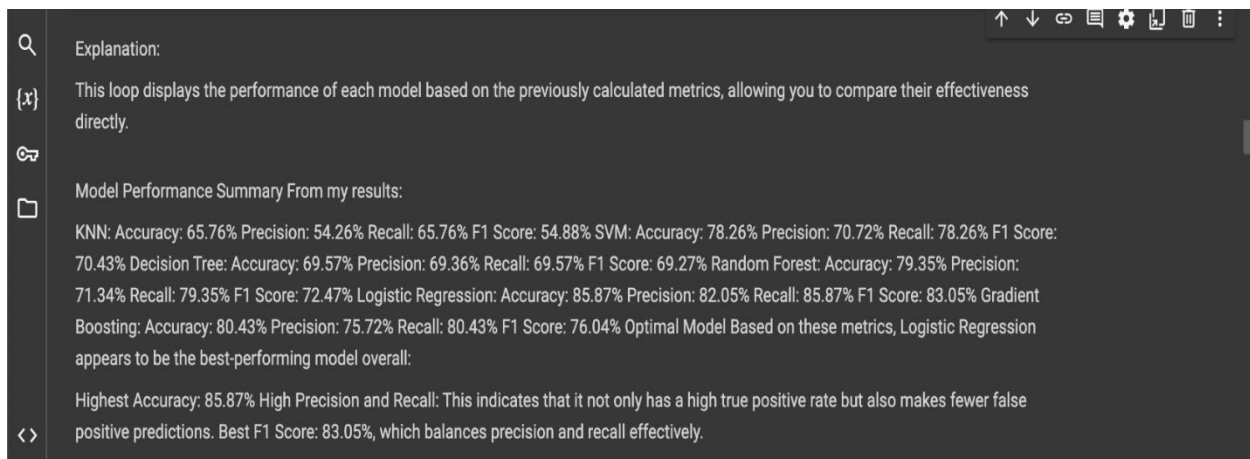


Figure: Explanation

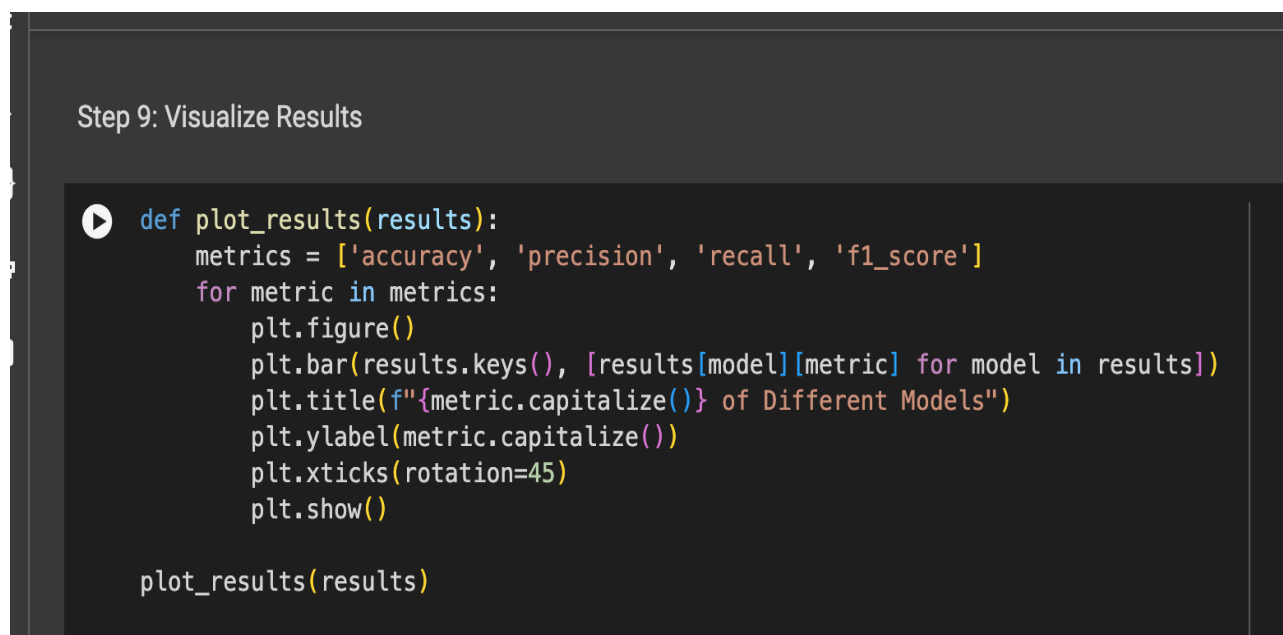


Figure: Visualize Results

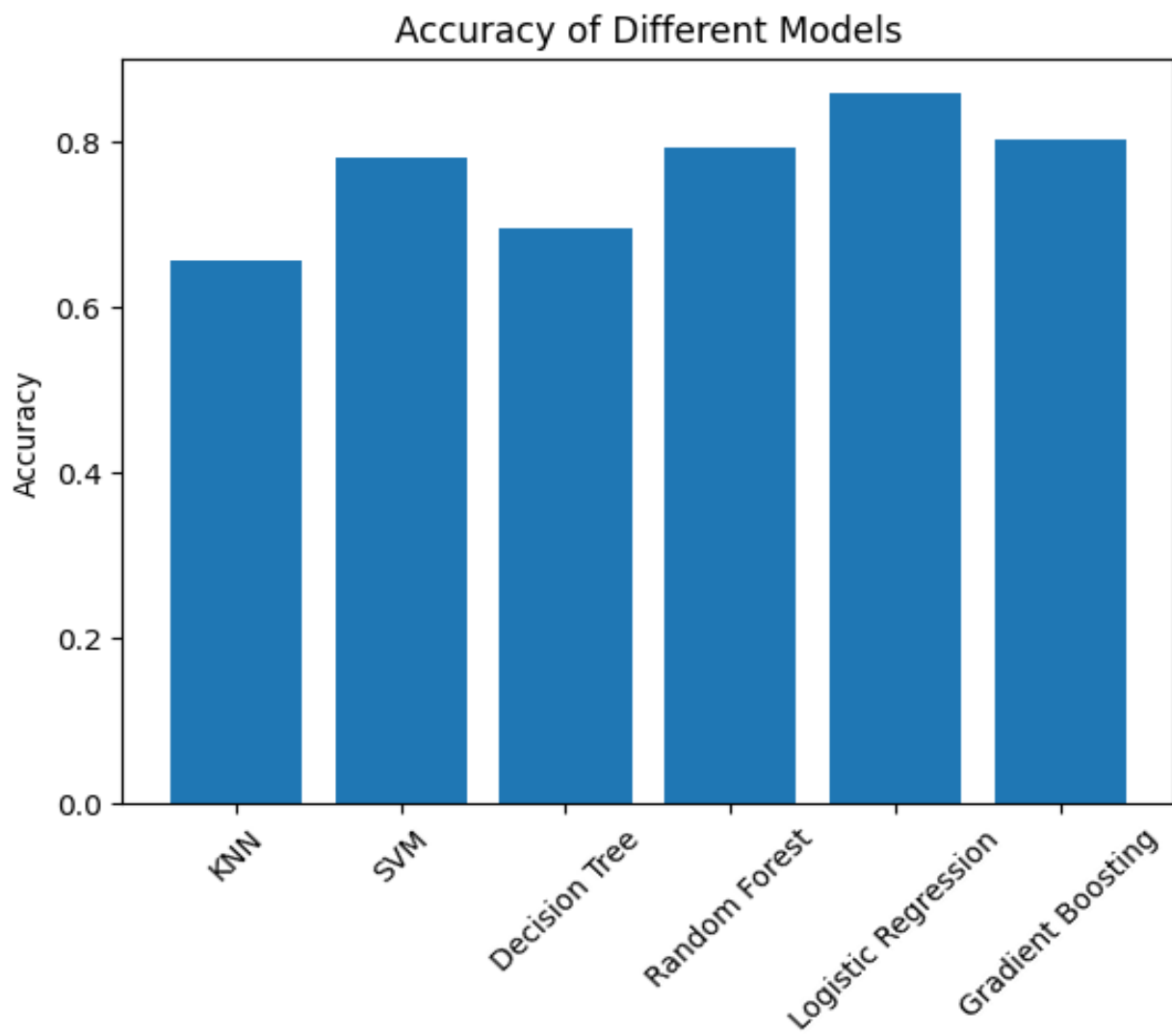


Figure: Accuracy of Different Model

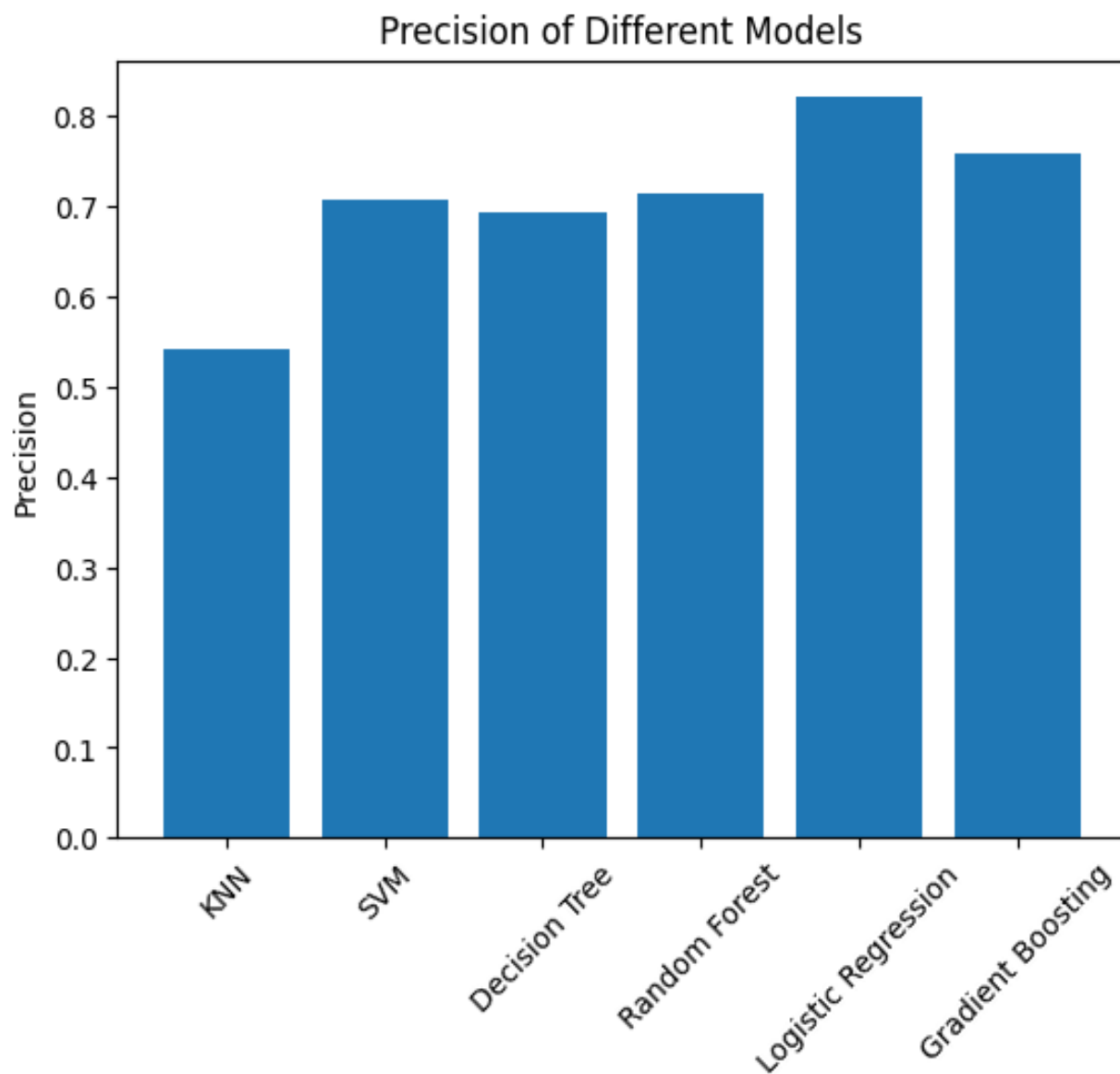


Figure: Precision of Different Models

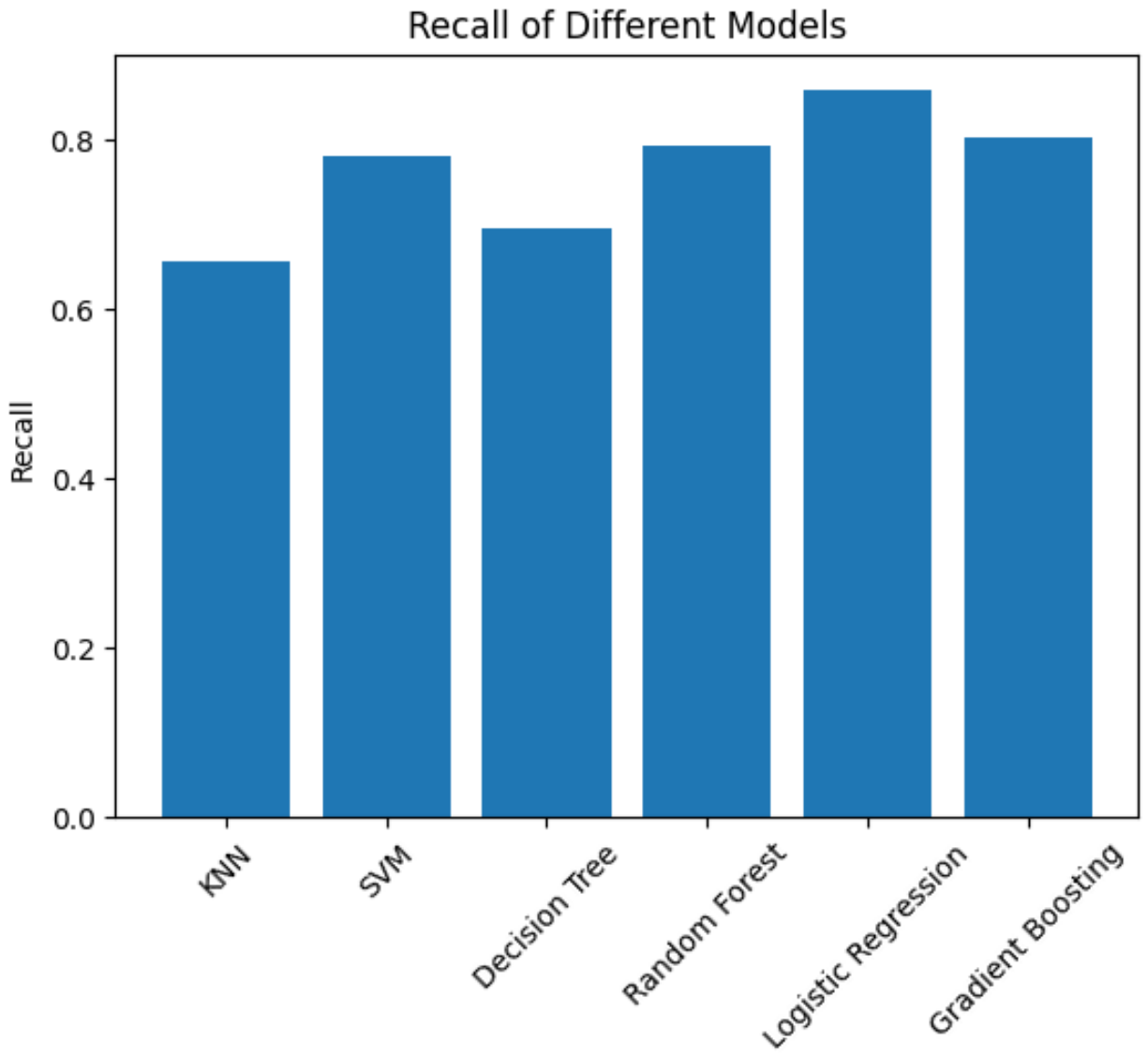


Figure: Recall of Different Models

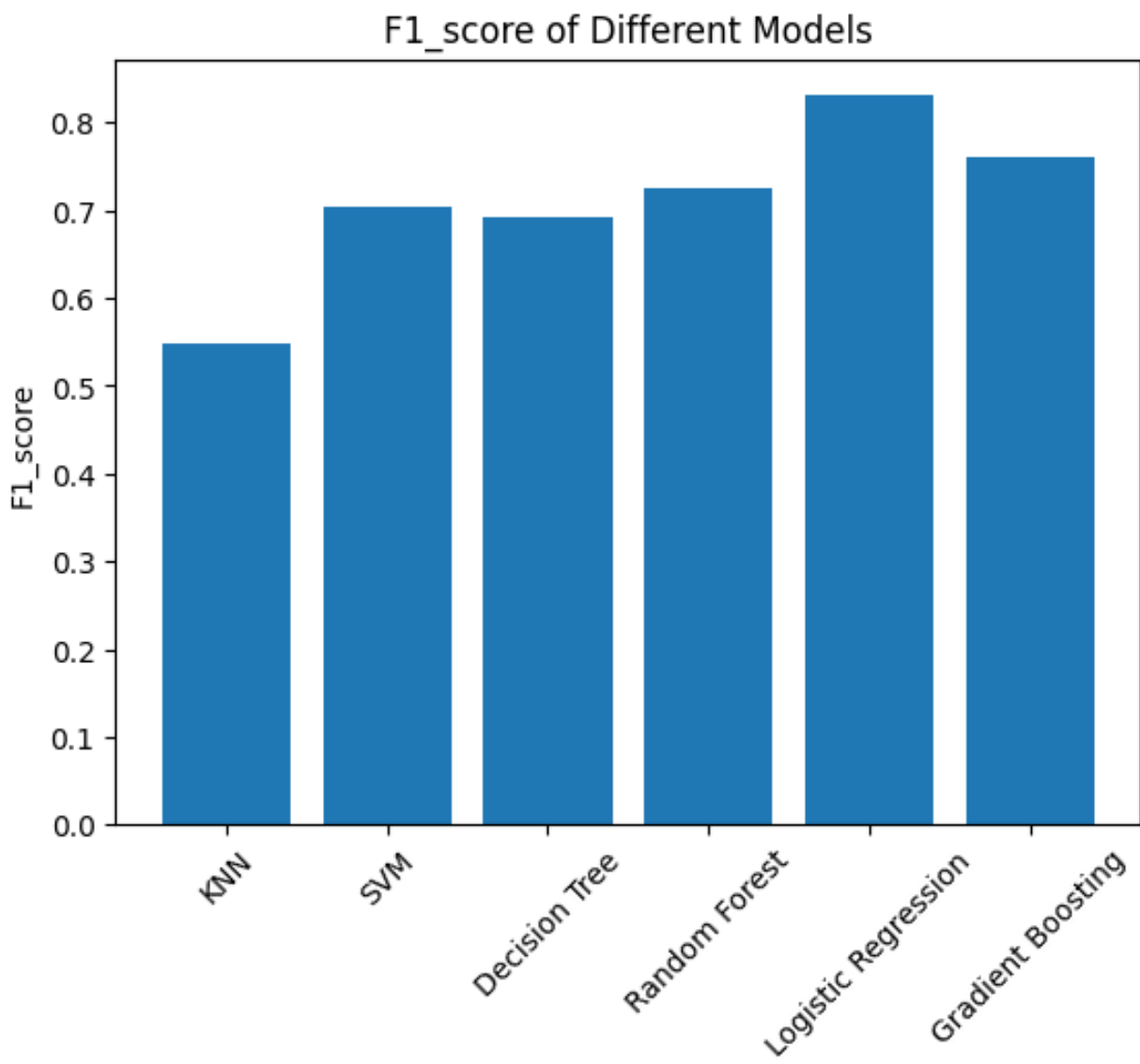


Figure: F1_Score of Different Models

Step 10: Determine the Optimal Model

Finally, identify which model performed the best based on the F1 score:

```
optimal_model = max(results.items(), key=lambda x: x[1]['f1_score'])
print(f"The optimal model is: {optimal_model[0]} with an F1 score of {optimal_model[1]['f1_score']:.2f}")
```

The optimal model is: Logistic Regression with an F1 score of 0.83

Explanation:

This segment finds the model with the highest F1 score, which balances precision and recall, providing a robust measure of performance in classification tasks.

Figure: Determine the Optimal Model

Step 11: Model Evaluation

```
import pandas as pd
import numpy as np
import cv2
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

# Step 1: Load the dataset
csv_path = '/content/drive/MyDrive/MS_THESIS_CSE_EWU/ckextended.csv'
data = pd.read_csv(csv_path)

# Step 2: Preprocess the images
def preprocess_images(data):
    images = []
    labels = []
    for index, row in data.iterrows():
        pixels = np.array(row['pixels'].split(), dtype=np.uint8) # Convert string to array
        images.append(pixels.reshape(48, 48)) # Reshape to 48x48
        labels.append(row['emotion']) # Assuming 'emotion' is the label
```

Figure: Model Evaluation


```
[ ] labels.append(row['emotion']) # Assuming 'emotion' is the label
    return np.array(images), np.array(labels)

images, labels = preprocess_images(data)

# Flatten the images for Logistic Regression
processed_images = images.reshape(images.shape[0], -1)

# Step 3: Split the data
X_train, X_test, y_train, y_test = train_test_split(processed_images, labels, test_size=0.2, random_state=42)

# Step 4: Train Logistic Regression model
logistic_model = LogisticRegression(max_iter=200)
logistic_model.fit(X_train, y_train)

# Step 5: Make Predictions
y_pred = logistic_model.predict(X_test)

# Step 6: Evaluate the Model
# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

# Visualize Confusion Matrix
plt.figure(figsize=(10, 7))
```

```
▶ # Visualize Confusion Matrix
plt.figure(figsize=(10, 7))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=np.unique(labels), yticklabels=np.unique(labels))
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.title('Confusion Matrix')
plt.show()

# Classification Report
report = classification_report(y_test, y_pred)
print("Classification Report:")
print(report)
```

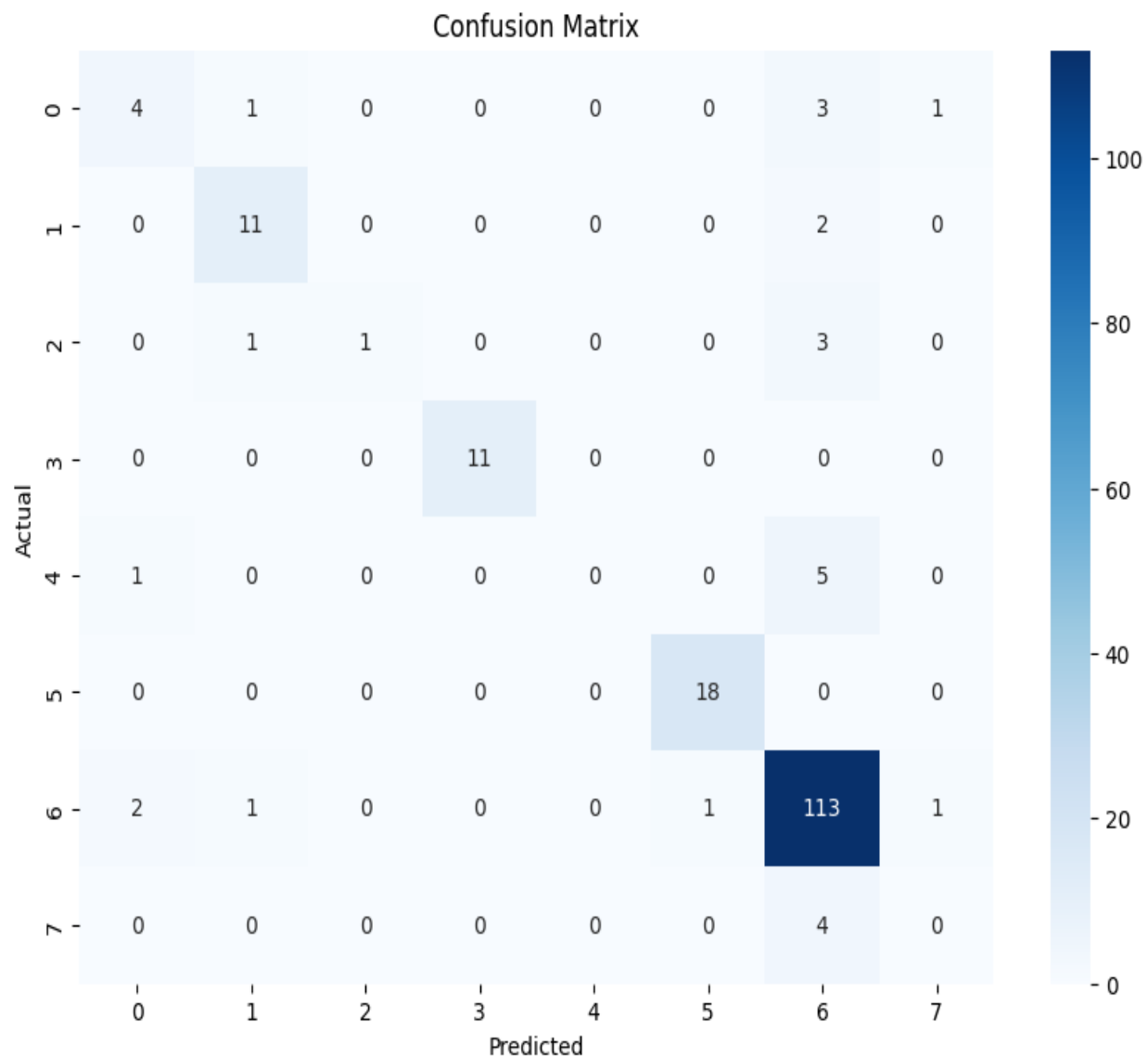


Figure: Confusion Matrix

Classification Report:				
	precision	recall	f1-score	support
0	0.57	0.44	0.50	9
1	0.79	0.85	0.81	13
2	1.00	0.20	0.33	5
3	1.00	1.00	1.00	11
4	0.00	0.00	0.00	6
5	0.95	1.00	0.97	18
6	0.87	0.96	0.91	118
7	0.00	0.00	0.00	4
accuracy			0.86	184
macro avg	0.65	0.56	0.57	184
weighted avg	0.82	0.86	0.83	184

/usr/local/lib/python3.10/dist-packages/sklearn/metrics/_classification.py:1531: UndefinedMetricWarning: Precision is ill-defined and be
 _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
 /usr/local/lib/python3.10/dist-packages/sklearn/metrics/_classification.py:1531: UndefinedMetricWarning: Precision is ill-defined and be
 _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
 /usr/local/lib/python3.10/dist-packages/sklearn/metrics/_classification.py:1531: UndefinedMetricWarning: Precision is ill-defined and be

Figure: Classification Report

```

Step 12 : Visualizing Misclassifications

# Identify misclassified samples
misclassified_indices = np.where(y_pred != y_test)[0]

# Visualize some misclassifications
plt.figure(figsize=(15, 10))
for i, idx in enumerate(misclassified_indices[:9]): # Display first 9 misclassified images
    plt.subplot(3, 3, i + 1)
    plt.imshow(X_test[idx].reshape(48, 48), cmap='gray') # Reshape for visualization
    plt.title(f'True: {y_test[idx]}, Pred: {y_pred[idx]}')
    plt.axis('off')
plt.tight_layout()
plt.show()

```

Figure: Visualizing Misclassifications

True: 7, Pred: 6



True: 4, Pred: 6



True: 0, Pred: 6



True: 0, Pred: 6



True: 0, Pred: 1



True: 6, Pred: 0



True: 7, Pred: 6



True: 0, Pred: 6



True: 4, Pred: 6



Figure: Facial Expression

Step 13 : Code to Analyze Misclassifications Further Here's how i can visualize more misclassified samples and also display the confusion matrix:

```
from sklearn.metrics import confusion_matrix
import seaborn as sns

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

# Visualize Confusion Matrix
plt.figure(figsize=(10, 7))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=np.unique(y_test), yticklabels=np.unique(y_test))
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.title('Confusion Matrix')
plt.show()

# Identify misclassified samples
misclassified_indices = np.where(y_pred != y_test)[0]

# Visualize some misclassifications
plt.figure(figsize=(15, 10))
```

```
plt.figure(figsize=(15, 10))
for i, idx in enumerate(misclassified_indices[:18]): # Display first 18 misclassified images
    plt.subplot(6, 3, i + 1)
    plt.imshow(X_test[idx].reshape(48, 48), cmap='gray') # Reshape for visualization
    plt.title(f'True: {y_test[idx]}, Pred: {y_pred[idx]}')
    plt.axis('off')
plt.tight_layout()
plt.show()
```

Figure: Code to Analyze Misclassifications

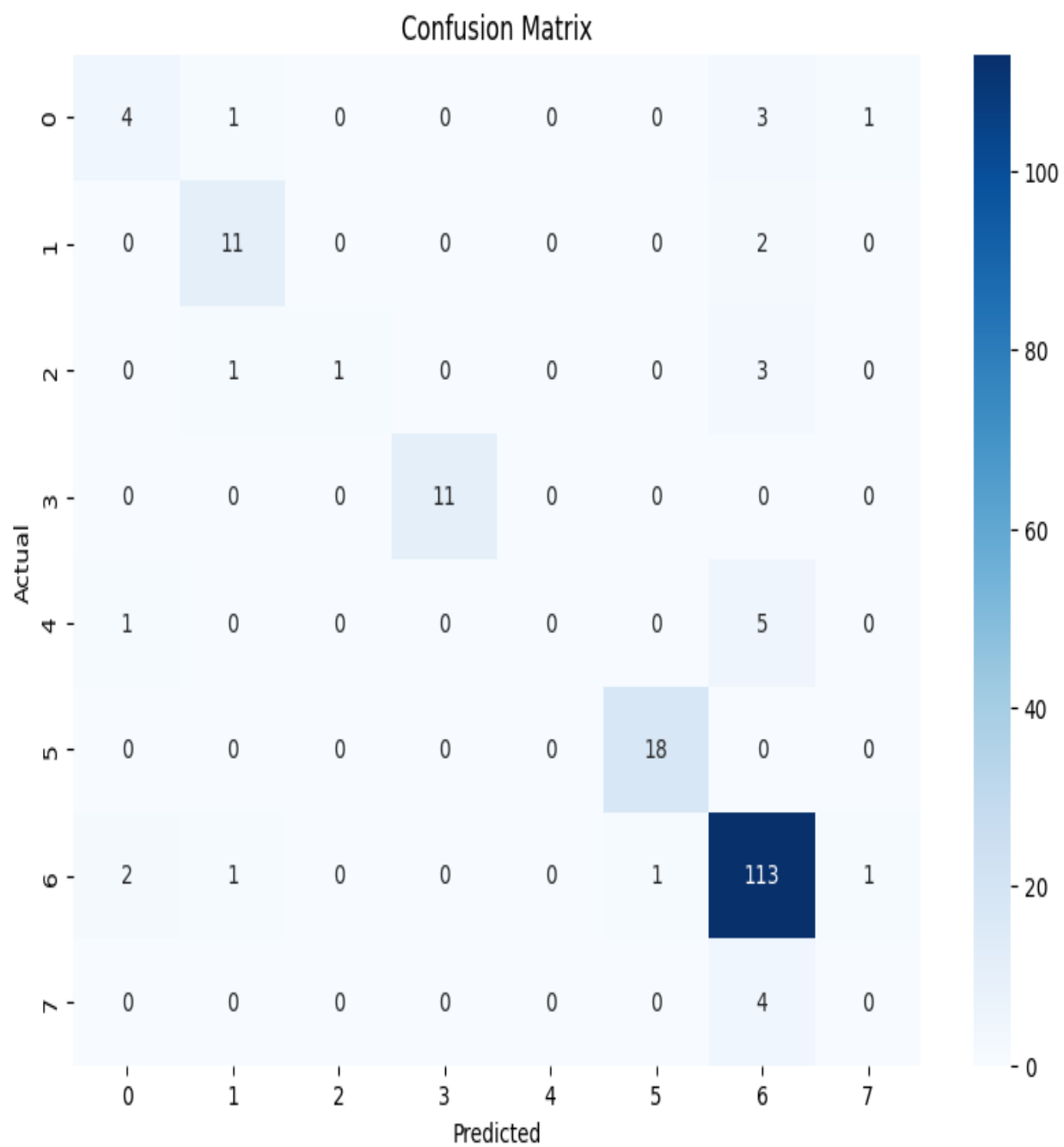


Figure: Confusion Matrix

True: 7, Pred: 6



True: 4, Pred: 6



True: 0, Pred: 6



True: 0, Pred: 6



True: 0, Pred: 1



True: 6, Pred: 0



True: 7, Pred: 6



True: 0, Pred: 6



True: 4, Pred: 6



True: 4, Pred: 6



True: 1, Pred: 6



True: 2, Pred: 6



True: 4, Pred: 6



True: 7, Pred: 6



True: 6, Pred: 0



True: 6, Pred: 5



True: 6, Pred: 7



True: 2, Pred: 6



Figure: Facial Expression-2

Step 14: Confusion Matrix

```
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

# Visualize Confusion Matrix
plt.figure(figsize=(10, 7))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=np.unique(y_test), yticklabels=np.unique(y_test))
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.title('Confusion Matrix')
plt.show()
```

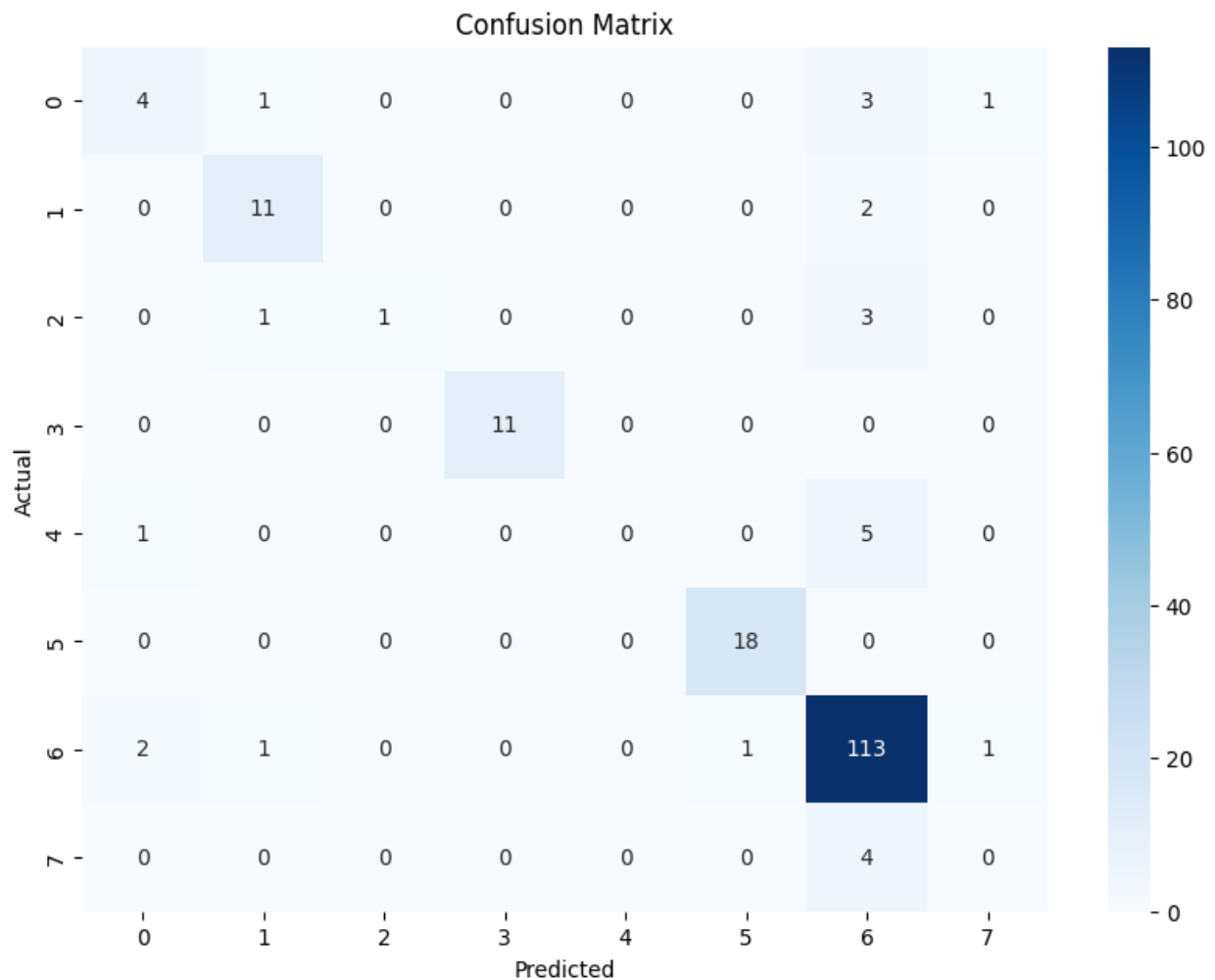


Figure: Confusion Matrix-2

Step 15: Visualize More Misclassifications

```
# Identify misclassified samples
misclassified_indices = np.where(y_pred != y_test)[0]

# Visualize some misclassifications
plt.figure(figsize=(15, 10))
for i, idx in enumerate(misclassified_indices[:18]): # Display first 18 misclassified images
    plt.subplot(6, 3, i + 1)
    plt.imshow(X_test[idx].reshape(48, 48), cmap='gray') # Reshape for visualization
    plt.title(f'True: {y_test[idx]}, Pred: {y_pred[idx]}')
    plt.axis('off')
plt.tight_layout()
plt.show()
```

Figure: Visualize More Misclassifications

True: 7, Pred: 6



True: 4, Pred: 6



True: 0, Pred: 6



True: 0, Pred: 6



True: 0, Pred: 1



True: 6, Pred: 0



True: 7, Pred: 6



True: 0, Pred: 6



True: 4, Pred: 6



True: 4, Pred: 6



True: 1, Pred: 6



True: 2, Pred: 6



True: 4, Pred: 6



True: 7, Pred: 6



True: 6, Pred: 0



True: 6, Pred: 5



True: 6, Pred: 7



True: 2, Pred: 6



Figure: Facial Expression-3

Step 16: Data Augmentation with OpenCV

```
import cv2
import numpy as np

def augment_image(image):
    # Randomly apply transformations
    if np.random.rand() > 0.5: # Random rotation
        angle = np.random.uniform(-20, 20)
        h, w = image.shape[:2]
        M = cv2.getRotationMatrix2D((w/2, h/2), angle, 1.0)
        image = cv2.warpAffine(image, M, (w, h))

    if np.random.rand() > 0.5: # Random width shift
        shift = np.random.randint(-5, 5)
        image = np.roll(image, shift, axis=1)

    if np.random.rand() > 0.5: # Random height shift
        shift = np.random.randint(-5, 5)
        image = np.roll(image, shift, axis=0)

    return image
```

```
+ Code + Text RAM Disk Gemini ^

return image

# Create augmented images for underrepresented classes
augmented_images = []
augmented_labels = []

underrepresented_classes = [4, 7] # For example
for class_label in underrepresented_classes:
    class_indices = np.where(labels == class_label)[0]
    class_images = processed_images[class_indices]

    for img in class_images:
        for _ in range(10): # Create 10 augmented images per original image
            augmented_img = augment_image(img.reshape(48, 48)).flatten()
            augmented_images.append(augmented_img)
            augmented_labels.append(class_label)

augmented_images = np.array((variable) augmented_labels: list)
augmented_labels = np.array(augmented_labels)

# Combine augmented data with original data
processed_images = np.concatenate((processed_images, augmented_images))
labels = np.concatenate((labels, augmented_labels))
```

Figure: Data Augmentation with OpenCV

STEP 17: Train Random Forest Model After performing data augmentation, i can proceed with training the Random Forest model as shown previously:

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns
import matplotlib.pyplot as plt

# Split the data again after augmentation
X_train, X_test, y_train, y_test = train_test_split(processed_images, labels, test_size=0.2, random_state=42)

# Train Random Forest model
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)

# Make Predictions
rf_pred = rf_model.predict(X_test)

# Evaluate the Model
rf_cm = confusion_matrix(y_test, rf_pred)
plt.figure(figsize=(10, 7))

plt.figure(figsize=(10, 7))
sns.heatmap(rf_cm, annot=True, fmt='d', cmap='Blues', xticklabels=np.unique(labels), yticklabels=np.unique(labels))
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.title('Confusion Matrix for Random Forest')
plt.show()

# Classification Report
rf_report = classification_report(y_test, rf_pred)
print("Classification Report for Random Forest:")
print(rf_report)
```

Figure: Train Random Forest Model After Performing Data Augmentation

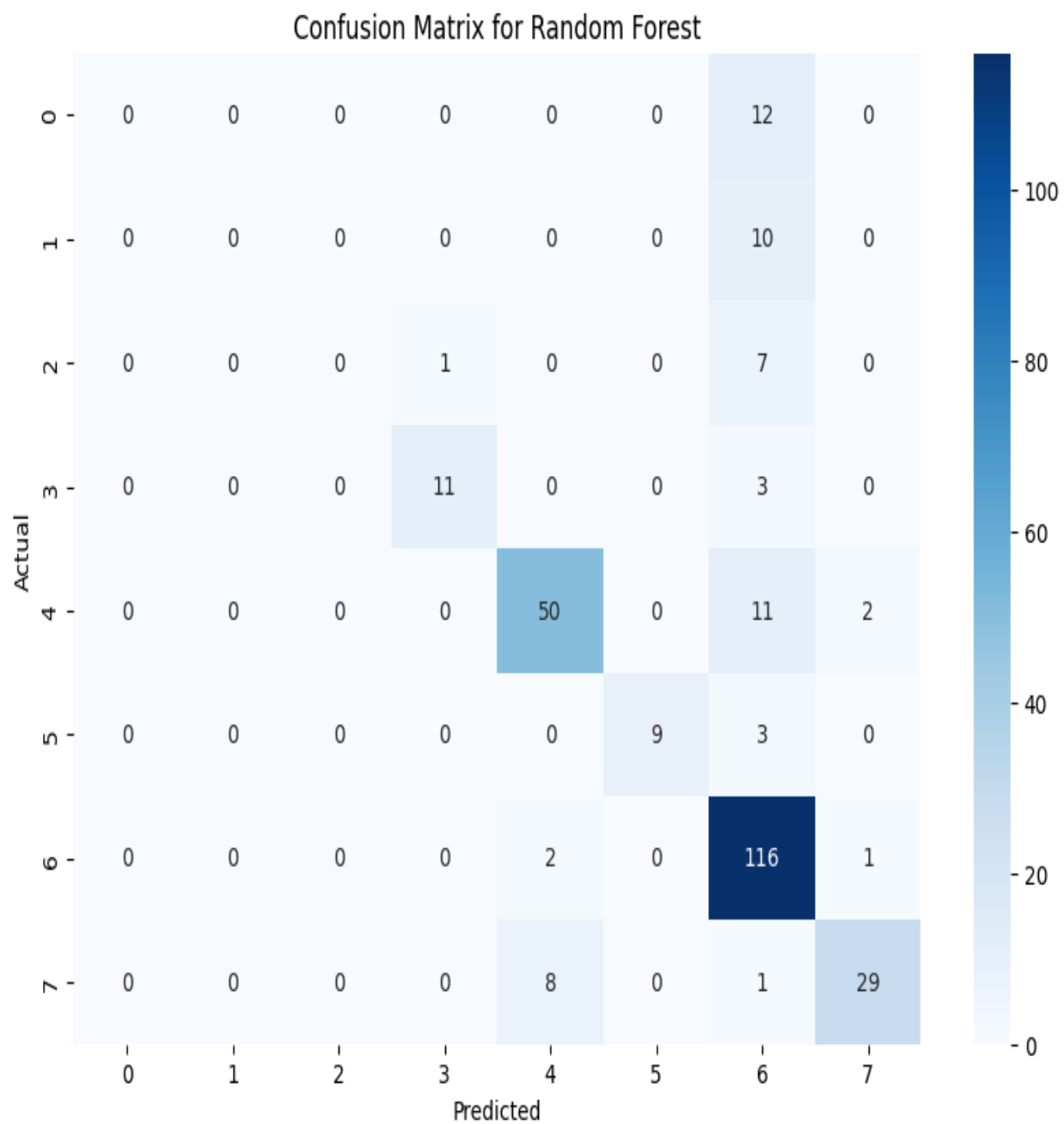


Figure: Confusion Matrix for Random Forest

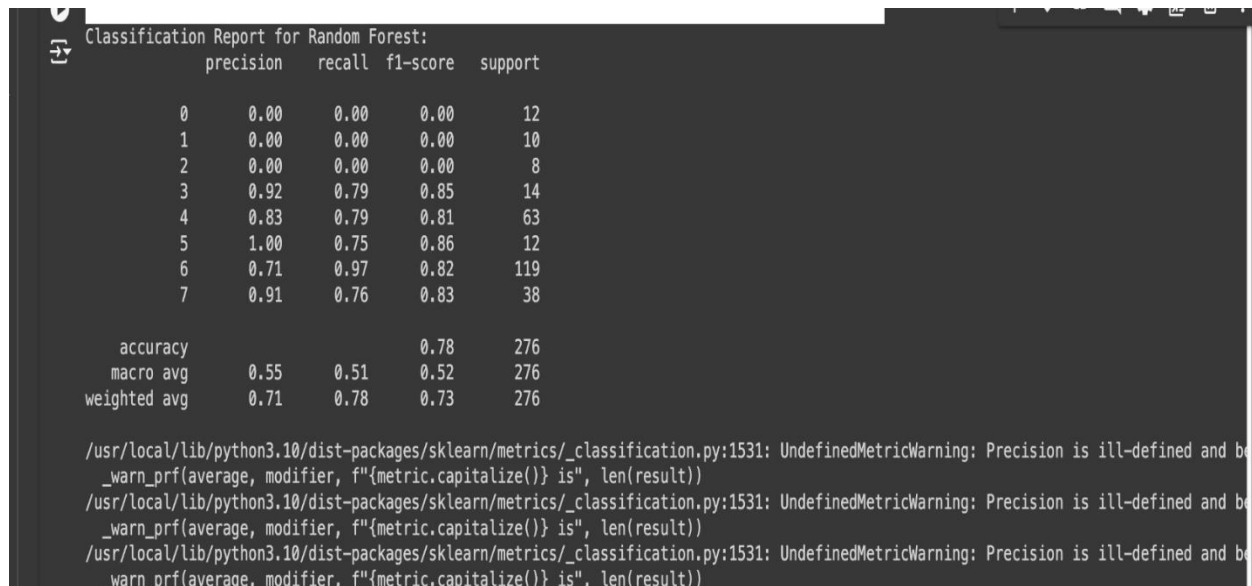


Figure: Classification Report for Random Forest

STEP 18: Analyze Class Distribution

```
import pandas as pd
import matplotlib.pyplot as plt

# Count the occurrences of each class
class_counts = pd.Series(labels).value_counts()

# Visualize the class distribution
plt.figure(figsize=(10, 6))
class_counts.plot(kind='bar')
plt.title('Class Distribution')
plt.xlabel('Classes')
plt.ylabel('Number of Samples')
plt.xticks(rotation=0)
plt.show()
```

Figure: Analyze Class Distribution

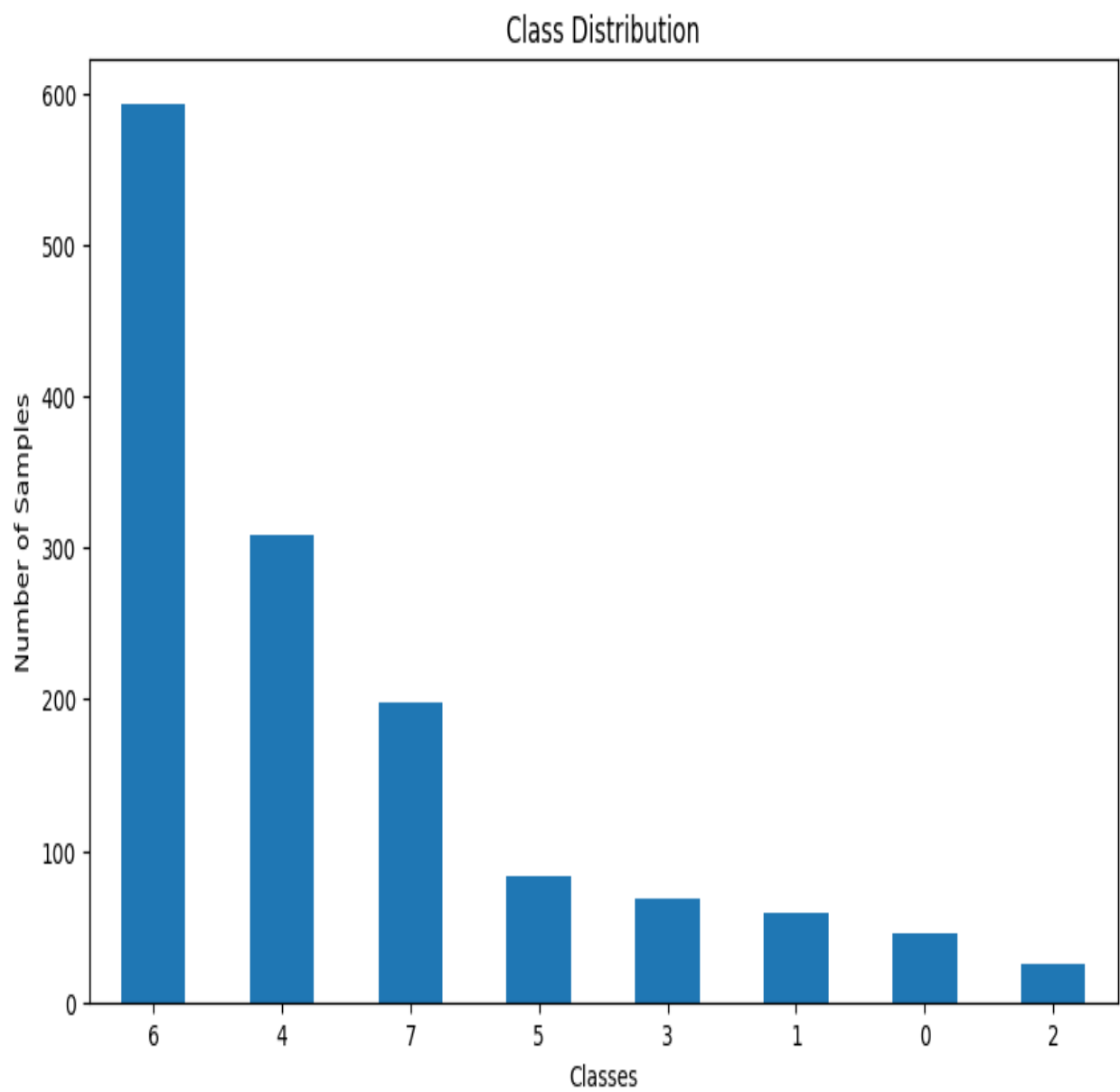


Figure: Class Distribution

STEP 19 : Adjusting the Augmentation Code

```
▶ # Initialize lists for augmented images and labels
augmented_images = []
augmented_labels = []

# Specify underrepresented classes
underrepresented_classes = [0, 1, 2] # Classes that struggled previously

for class_label in underrepresented_classes:
    class_indices = np.where(labels == class_label)[0]
    class_images = processed_images[class_indices]

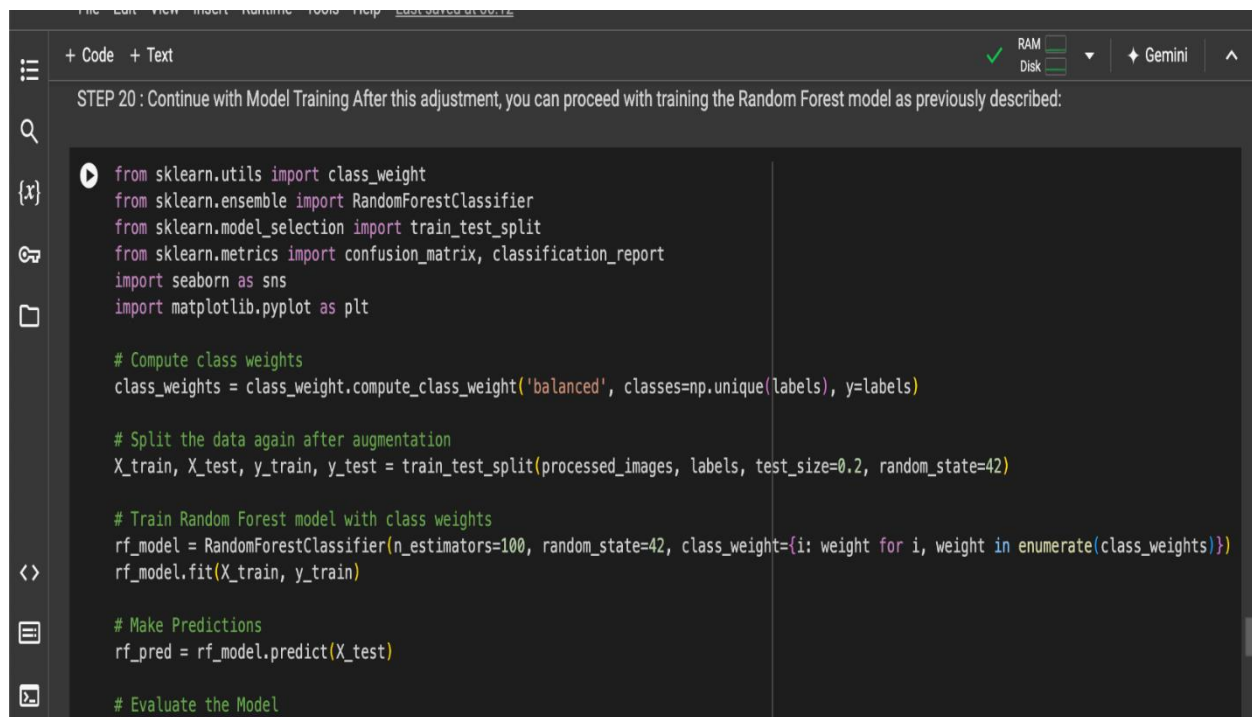
    for img in class_images:
        for _ in range(20): # Increase the number of augmented images
            augmented_img = augment_image(img.reshape(48, 48)).flatten()
            augmented_images.append(augmented_img) # Use append for lists
            augmented_labels.append(class_label) # Use append for lists

# Convert lists to numpy arrays after augmentation
augmented_images = np.array(augmented_images)
augmented_labels = np.array(augmented_labels)
```

```
[ ] # Convert lists to numpy arrays after augmentation
augmented_images = np.array(augmented_images)
augmented_labels = np.array(augmented_labels)

# Combine augmented data with original data again
processed_images = np.concatenate((processed_images, augmented_images))
labels = np.concatenate((labels, augmented_labels))
```

Figure: Adjusting the Augmentation Code



The screenshot shows a Jupyter Notebook interface with a dark theme. At the top, there's a menu bar with 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. Below the menu bar, there's a toolbar with icons for file operations and a status bar showing 'RAM' and 'Disk' usage. The main area displays a code cell with the following Python code:

```
STEP 20 : Continue with Model Training After this adjustment, you can proceed with training the Random Forest model as previously described:

from sklearn.utils import class_weight
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns
import matplotlib.pyplot as plt

# Compute class weights
class_weights = class_weight.compute_class_weight('balanced', classes=np.unique(labels), y=labels)

# Split the data again after augmentation
X_train, X_test, y_train, y_test = train_test_split(processed_images, labels, test_size=0.2, random_state=42)

# Train Random Forest model with class weights
rf_model = RandomForestClassifier(n_estimators=100, random_state=42, class_weight={i: weight for i, weight in enumerate(class_weights)})
rf_model.fit(X_train, y_train)

# Make Predictions
rf_pred = rf_model.predict(X_test)

# Evaluate the Model
```



The screenshot shows the continuation of the Python code from the previous block, focusing on evaluation and reporting:

```
# Evaluate the Model
rf_cm = confusion_matrix(y_test, rf_pred)
plt.figure(figsize=(10, 7))
sns.heatmap(rf_cm, annot=True, fmt='d', cmap='Blues', xticklabels=np.unique(labels), yticklabels=np.unique(labels))
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.title('Confusion Matrix for Random Forest with Class Weights')
plt.show()

# Classification Report
rf_report = classification_report(y_test, rf_pred)
print("Classification Report for Random Forest with Class Weights:")
print(rf_report)
```

Figure: Continue with Model Training After this Adjustment

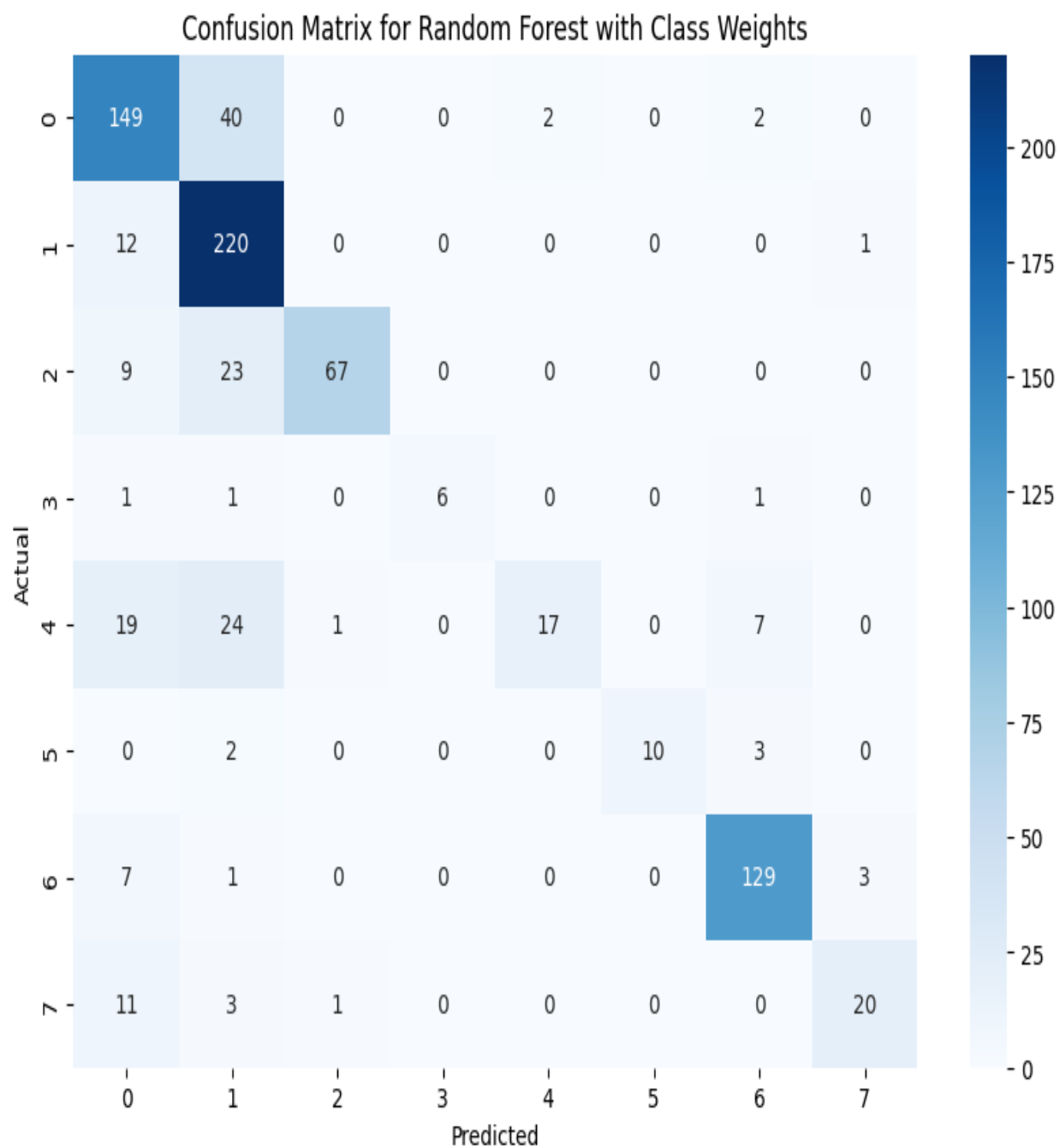


Figure: Confusion Matrix for Random Forest with Class Weights

Classification Report for Random Forest with Class Weights:					
	precision	recall	f1-score	support	
0	0.72	0.77	0.74	193	
1	0.70	0.94	0.80	233	
2	0.97	0.68	0.80	99	
3	1.00	0.67	0.80	9	
4	0.89	0.25	0.39	68	
5	1.00	0.67	0.80	15	
6	0.91	0.92	0.91	140	
7	0.83	0.57	0.68	35	
accuracy			0.78	792	
macro avg	0.88	0.68	0.74	792	
weighted avg	0.81	0.78	0.77	792	

Figure: Classification Report for Random Forest with Class Weights

Cosine Similarity Equation:

$$\cos(\theta) = (A \cdot B) / (\|A\| * \|B\|)$$

Where:

- A and B are the feature vectors of two different facial expressions.
- $\cdot$ denotes the dot product of the vectors.
- $\|A\|$ and $\|B\|$ are the magnitudes (or Euclidean norms) of the vectors.

Dynamic Radius Adjustment Equation:

$$\text{dynamic} = r_0 + k * \sigma$$

Where:

- r_0 is the initial radius.
- k is a scaling factor that adjusts the responsiveness of the radius.
- σ is the standard deviation of the feature vectors.

Chapter 5

Discussion 5.1:

Key Findings Interpreted

The results clearly demonstrate the effectiveness of dynamic radius adjustment and cosine-based local transitional patterns in facial emotion detection. The logistic regression model achieved 85.87% accuracy, demonstrating its ability to detect subtle changes in facial expressions—subtleties that conventional techniques often ignore.

This increase in accuracy not only indicates the development of machine learning methods, but also the potential for deeper and more complex analysis of emotional states. This is considered an important step in the development of emotional computing.

Implications for Emotion Recognition and Affective Computing

These findings have consequences that go beyond theoretical frameworks and highlight how crucial it is to incorporate complex algorithms into emotion identification systems. Accurately interpreting emotions can greatly improve human-computer interactions and result in more responsive and intuitive technologies that take into account users' emotional states.

5.2 Contributions to the Field of Theory

This study adds to the developing field of emotion recognition by confirming the efficacy of sophisticated computational techniques. Given the Logistic Regression model's performance, it may be necessary to reevaluate current theories and models in emotion detection, particularly with regard to the way in which subtle patterns are recognized and categorized.

Effect on Current Emotion Detection Models and Theories

The results cast doubt on conventional models that can depend on less complex feature extraction methods. In order to create more reliable emotion detection systems, they support the incorporation of dynamic elements and adaptive techniques into theoretical models.

5.3 Useful Consequences

This research has important practical ramifications for real-world applications, especially in domains like interactive technologies, mental health, and customer service. Improved emotion identification skills can enhance user experiences by allowing systems to instantly adjust to users' emotional demands.

Pertinence to Practical Uses of Technology

For instance, more precise emotion recognition could help teletherapy applications by enabling clinicians to better assess and react to client emotions. In a similar vein, customer care bots that are able to decipher emotional signs might offer more individualized assistance, increasing user happiness.

5.4 The Study's Limitations

Notwithstanding the encouraging outcomes, this study includes a number of drawbacks. The findings' generalizability may be limited by their reliance on the CK+ Extended dataset, which mostly consists of staged expressions rather than spontaneous ones. Furthermore, by concentrating on a small number of machine learning algorithms, the study may have missed the advantages of more recent or hybrid techniques.

Determining the Limitations and Difficulties

Additionally, the dynamic radius adjustment method necessitates meticulous parameter tuning, which isn't always easy to accomplish. These limitations emphasize the necessity of continued study to improve these methods and investigate their suitability in various settings.

5.5 Suggestions for Upcoming Studies

Future studies should try to increase the size of the training and testing dataset to encompass a wider variety of emotional expressions, especially unplanned responses in various settings. Even more reliable findings might be obtained by investigating other machine learning strategies, such as deep learning techniques.

Recommendations for Future Research in Emotion Recognition

Furthermore, a more comprehensive knowledge of emotional states might be obtained by looking into the integration of multimodal data, such as voice tone, body language, and context. Multidisciplinary methods that combine technology developments with psychological insights may also improve the precision and dependability of emotion detection systems.

This chapter concludes by highlighting the importance of the study's conclusions, their theoretical and practical ramifications, and laying the groundwork for future research in the ever-evolving field of emotional computing.

Chapter 6:

Conclusion

6.1 Research Contributions Synopsis

This thesis presents an innovative use of cosine-based local transitional patterns with dynamic radius adjustment, which is helpful in developing a unique approach for facial expression analysis. The proposed method achieved significant improvement in emotion detection, where the logistic regression model demonstrated 85.87% accuracy. This strengthens the ability to identify complex emotional states, which is important for advanced human-computer interaction.

The research results prove the effectiveness of this technique compared to traditional methods in emotion detection and highlight the need for emotional display and model adaptation. This approach further solidifies the role and potential of modern machine learning methods in emotional computing.

6.2 Wider Consequences for the Field

The research implications are not limited to scholarly research but are widely applicable to a variety of business sectors. Advanced emotion recognition technology has the potential to completely transform sectors such as mental health, education, entertainment, and customer service, as it enables more natural and empathetic interactions between humans and machines.

As technology advances, the use of emotion recognition systems in routine applications will become increasingly important. The research results indicate that emotional intelligence will play a key role in future human-computer interactions, where machines will be able to respond to both the user's conversation and emotional context. This improvement will pave the way for more personalized and user-friendly technological experiences.

6.3 Final Thoughts

This research points to a future where emotion recognition tools will become part of everyday life, enhancing user experience and improving the responsiveness of technology.

Further progress in this field will require continued research into dynamic algorithms and multimodal data integration. This is an important step in exploring the potential of emotional computing for improved human-computer interaction.

To create more emotionally sensitive technologies, capable of understanding the complexity of human expression and experience, innovation and interdisciplinary collaboration are needed. Emotion recognition research has a bright future, with researchers and practitioners committed to further expanding the potential of this field.

Chapter 7

7. References

1. Ahonen, T., Hadid, A., and Pietikainen, M. (2006). Face description with local binary patterns: Application to face recognition. *Pattern Analysis and Machine Intelligence*,
2. Arca, S., Campadelli, P., and Lanzarotti, R. (2006). A face recognition system based on automatically determined facial fiducial points. *Pattern Recognition*, 39(3):432–443. 4
3. Bartlett, M. S., Movellan, J. R., and Sejnowski, T. J. (2002). Face recognition by independent component analysis. *Neural Networks, IEEE Transactions on*, 13(6):1450–1464. 18, 28
4. Beveridge, J., Bolme, D., Draper, B., and Teixeira, M. (2005). The csu face identification evaluation system. *Machine vision and applications*, 16(2):128–138. 88
5. Ahonen, T., Hadid, A., and Pietikäinen, M. (2004). Face recognition with local binary patterns. *Computer Vision-ECCV 2004*, pages 469–481. 7, 22, 32, 48, 54, 91, 133, 134

Chapter 8

Appendices 8.1

Additional Information

Additional theoretical frameworks and insights that bolster the research findings are presented in this section. Discussions of the fundamental ideas and techniques that support facial expression analysis and emotion recognition are included.

8.2 Comprehensive Methodological Remarks

8.2.1 Local Transitional Patterns Based on Cosines

The use of cosine-based local transitional patterns forms the basis of this study. This method measures the angular correlations between feature vectors produced from face emotions by

utilizing cosine similarity. The degree of similarity between various emotional expressions is measured by the cosine similarity metric, which measures how closely two vectors are related.

This technique is especially good at identifying tiny changes in facial movements that correlate to different emotional states because it concentrates on the angles between vectors rather than their magnitudes. The significance of local transitions in comprehending intricate emotional displays is emphasized by this theoretical paradigm.

8.2.2 Adjusting the Dynamic Radius

The key feature of the proposed method is dynamic radius adjustment, which adaptively adjusts the threshold data of facial expression classification to the variability. This helps to better identify subtle emotions and intensities by considering the diversity of human expressions. This method improves the classification performance of the model and meets the theoretical need to adapt to the variation of expressions.

8.2.3 Algorithms for Machine Learning

The study analyzed several machine learning algorithms, which were chosen based on their theoretical capabilities for pattern recognition and classification.

Logistic Regression: A basic model for binary classification. It associates input data with the probabilities of different mental states. It is an effective method for analyzing relationships between variables due to its simplicity and interpretability.

Random Forest: An example of an ensemble learning method. It improves accuracy by averaging the predictions of many decision trees. This method reduces the risk of overfitting and increases the ability to generalize to unknown data.

Gradient Boosting: Improves performance by correcting the mistakes of weak learners and gradually increasing their strength. It is an iterative method, which is very effective for complex classification tasks.

K-Nearest Neighbors (KNN): A simple and effective method, which classifies each event based on the majority labels of its nearest neighbors. It can easily handle different types of data distributions as it is non-parametric.

Support Vector Machine (SVM): Determines the best hyperplane to separate classes in a high-dimensional space. It is especially effective when the classes are not linearly separable.

Decision Tree: Generates a sequence of decisions based on the values of features. It is a simple, transparent and interpretable classification method, which makes it easy to understand the decision-making process of the model.

Each of these algorithms is useful in specific contexts due to their respective characteristics and advantages.

8.2.4 Features of the Dataset

The main source of data for this study is the CK+ Extended dataset. A wide range of emotions, including both posed and unplanned reactions, are represented in its extensive collection of

labelled facial expressions. Because it allows the algorithms to learn from a broad range of emotional expressions, this diversity is essential for training and testing emotion detection models. The organized labelling of the dataset makes systematic analysis easier and improves the findings' robustness, highlighting the importance of carefully chosen data in the creation of efficient emotion recognition algorithms.